

EmailBench: A Benchmark for Evaluating LLM Agents on Enterprise Email and Productivity Tasks

Mukul Singh Mansi Uniyal Devin Devlin Wen Xie Big Thadawasin
Ritam Dutt Vivian Lai Hyeonsu B. Kang

Microsoft

{singhmukul}@microsoft.com

Abstract

Enterprise email agents must combine information retrieval, structured state changes, temporal reasoning, and multi-step coordination. Recent agent benchmarks include productivity tasks, but few center on typed email workflows in a self-contained environment. We introduce **EmailBench**, a benchmark of 206 email and productivity scenarios across 16 task categories. The benchmark couples a typed email API specification with provider-neutral naming, a deterministic synthetic Enron-inspired corpus, and a scenario suite whose topic selection was informed by aggregate task-intent telemetry from an interactive prototype. Its hybrid evaluation protocol combines 258 executable static assertions with 211 LLM rubrics. We evaluate eight LM configurations on a fixed single-user corpus. The best-performing configuration passes only 33.5% of scenarios despite 99.7% of its tool calls completing without an observed API failure, with pass rates varying substantially across task categories. This gap shows that valid tool execution is not equivalent to task completion. EmailBench¹ provides a self-contained environment for end-to-end email-agent evaluation, with broader tool coverage, multi-persona testing, and repeated-run evaluation as future work areas.

1 Introduction

Email agents operate over structured records rather than text alone. A request to schedule a meeting from an email thread may require message retrieval, participant resolution, availability checks, event creation, and a follow-up action. Each operation has a typed payload and later calls can depend on identifiers returned by earlier ones. An agent can therefore issue syntactically valid calls while still selecting the wrong records, violating temporal constraints, or stopping before the workflow is complete.

¹We plan to publicly release the benchmark and all supporting artifacts.

Existing agent benchmarks evaluate web navigation, desktop interaction, database operations, and repository-level software engineering (Liu et al., 2024; Zhou et al., 2024; Xie et al., 2024; Jimenez et al., 2024). Productivity-focused benchmarks evaluate workflows across office applications, including email and calendars (Wang et al., 2024; Trivedi et al., 2024; Wang et al., 2025; Li et al., 2026). Within this landscape, we focus on email-centered workflows that require agents to connect information across mailbox, calendar, contact, and task entities and carry out coordinated state changes.

We introduce **EmailBench**, a self-contained benchmark for evaluating these workflows through a typed API, a linked synthetic corpus, and complementary state-based and semantic checks. Our contributions are:

1. **A self-contained benchmark suite.** A typed email API specification with provider-neutral naming is paired with a deterministic synthetic Enron-inspired corpus and 206 scenarios across 16 task categories. Aggregate prototype intent frequencies influenced topic selection.
2. **A hybrid evaluation protocol.** We combine executable state and answer checks with scenario-specific LLM rubrics to assess task completion, using explicit score aggregation and critical-check requirements.
3. **An empirical analysis of email-agent reliability.** Across eight LM configurations, the highest observed task pass rate is 33.5%, despite high API-level execution success. Category-level comparisons and recorded trajectories reveal a gap between executing individual operations and completing the requested workflows.

Together, these contributions advance more realistic email-agent evaluation focused on workflow completion, not just API execution.

2 Related Work

Agent evaluation increasingly asks whether models can complete tasks through interaction, rather than merely produce correct standalone answers. This perspective spans heterogeneous environments (Liu et al., 2024), web-based tasks (Zhou et al., 2024; Yoran et al., 2024), desktop interaction (Xie et al., 2024), and repository-level software engineering (Jimenez et al., 2024). Across these settings, success depends on connecting intermediate actions to an externally evaluated outcome. EmailBench adopts this task-level perspective for structured email and productivity workflows.

Tool-use and productivity benchmarks examine complementary aspects of this challenge. Large API collections emphasize selecting and invoking appropriate tools (Qin et al., 2024; Patil et al., 2024), while office-automation and simulated-app environments place those operations within broader workflows (Wang et al., 2024; Trivedi et al., 2024). This distinction matters for email agents: an executable operation may still target the wrong entity or leave a requested workflow incomplete.

Recent enterprise and productivity benchmarks further emphasize cross-functional coordination, long-horizon execution, and stateful services (Vishwakarma et al., 2025; Wang et al., 2025; Li et al., 2026). Email and calendar tasks already appear in this literature, so their inclusion alone does not distinguish EmailBench. Instead, these benchmarks provide the closest context for its narrower focus on email-centered workflows and the relationships among mailbox, calendar, contact, and task entities.

Earlier email research provides complementary foundations in email corpora and message-content analysis (Klimt and Yang, 2004; Zhang et al., 2021). EmailBench shifts the evaluation target from understanding messages to acting on linked productivity records. Its contribution lies in combining a typed email-centered API, a fixed synthetic cross-entity corpus, 206 scenarios, and executable checks supplemented by semantic rubrics. This combination supports examining both individual operations and end-to-end task completion, rather than introducing stateful or semantic evaluation in isolation.

3 EmailBench

EmailBench consists of an API specification, a deterministic corpus, a scenario suite, and a hybrid grader. Table 1 summarizes the artifact and distinguishes the specification from the evaluated adapter.

Component	Scope
API specification	119 operational methods, 2 evaluation utilities, 36 typed interfaces
Evaluated adapter	46 tools spanning messages, calendar, contacts, todo, folders, filters, settings, and identity
Corpus	65 messages, 15 events, 15 contacts, 10 todo tasks, and linked productivity entities
Scenarios	206 tasks, 16 categories, 8 declared API domains, and three assigned difficulty levels
Grading	258 static assertions and 211 LLM rubrics

Table 1: EmailBench at a glance. The empirical study evaluates a subset of the API specification.

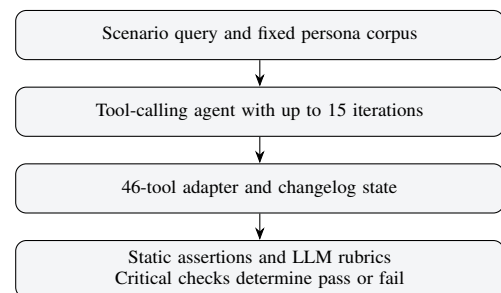


Figure 1: Evaluated pipeline. The adapter is narrower than the API specification, and all runs use the same persona corpus.

Detailed inventories appear in Appendix A, and complete scenario examples in Appendix L.

Figure 1 shows the evaluated path from a scenario query through the restricted adapter to the two grading components.

3.1 API and Corpus

The EmailBenchAPI specification defines 119 operational methods across ten domain objects plus identity. It also provides `reset()` and `snapshot()` for evaluation. The 36 interfaces use provider-neutral names and typed inputs and outputs. Portability across providers is a design goal, not an evaluated result. A snapshot exposes selected entity collections and mutation logs, which support deterministic checks after an agent finishes. The artifact includes an in-memory mutable implementation and a changelog implementation that reads from a frozen corpus and records writes.

The corpus is synthetic and is inspired by themes and organizational relationships in the public Enron email collection (Klimt and Yang, 2004). It does not reproduce source emails. The construction process extracts recurring business themes and

Category	<i>n</i>	Category	<i>n</i>
messages-read	39	todo	9
orchestration	17	meeting-prep	9
cross-domain	15	inbox-cleanup	9
messages-write	15	folders	9
multi-domain	12	calendar-triage	8
calendar-read	12	settings	8
calendar-write	12	contacts	12
boards	10	filters	10

Table 2: Distribution of the 206 scenarios. Categories denote task themes and are not a pure read-versus-write partition.

then generates 65 messages around a fixed reference time. Synthetic calendar events, contacts, todo tasks, boards, filters, and directory records extend the mailbox into a productivity environment. Cross-entity links connect senders to contacts and event attendees, and link project discussions to tasks and meetings. This design supports workflows that require joins across entity types while avoiding the release of private email content.

Evaluation environment. The backend pairs a fixed seed corpus with a separate typed action log, preserving agent operations for inspection and grading without modifying the seed data. Message reads remain tied to the original corpus rather than a fully updated mailbox view. Appendix O and Appendix Q detail the implications for workflows that verify earlier actions through subsequent reads.

3.2 Scenario Construction

The benchmark contains 206 scenarios in 16 thematic categories, summarized in Table 2. The initial task inventory was informed by aggregate user task-intent telemetry from a deployed prototype. Four construction batches added prototype failure cases, underrepresented API domains, additional named personas, and category coverage. The final suite contains 122 scenarios with one declared domain and 84 with two or more. Scenario authors assigned 17 tasks to D1, 121 to D2, and 68 to D3. These labels are metadata rather than experimentally calibrated difficulty levels, and the observed results are not monotonic across them.

Scenarios span retrieval, aggregation, entity creation, state mutation, and multi-step workflows. For example, a folder task creates a hierarchy and routes messages using the identifiers returned by folder-creation calls. A calendar workflow may retrieve participants from messages, identify an available interval, and create an event with nested

attendee and time-zone objects. Appendix L provides complete task and grading examples.

3.3 Evaluation Protocol

Our hybrid evaluation protocol assesses task completion using two complementary sources of evidence: executable static assertions and scenario-specific LLM rubrics. Static assertions check the final snapshot or textual answer, while LLM rubrics assess correctness, completeness, relevance, format, safety, action completion, and hallucination. Each scenario may use either component or both. The suite instantiates 258 static assertions across four of the nine supported types and contains 211 LLM rubrics; 43 scenarios have no static assertions. Within each component, assertion scores are combined as a weighted mean, with weights defaulting to one.

The static checks comprise 123 answer-substring checks (47.7%), 101 general predicates over the evaluation snapshot (39.1%), 32 existence checks (12.4%), and two checks on sent messages (0.8%). These checks verify specified conditions rather than exhaustively establish task completion; for example, confirming that a folder exists does not establish that all intended messages were moved into it. The 43 scenarios without static checks rely entirely on the LLM component. Appendix J reports the complete inventory, including five supported assertion types not instantiated in the evaluated suite.

We combine the two components into a scenario-level score, while critical assertions enforce requirements that cannot be offset by a higher score elsewhere. Let s_{static} and s_{LLM} denote the available component scores. When both components are present, the scenario score is

$$s = \frac{1}{2}s_{\text{static}} + \frac{1}{2}s_{\text{LLM}}. \quad (1)$$

When only one component is present, its score is used. A scenario passes when $s \geq 0.5$ and every assertion marked critical passes. A critical LLM assertion passes when its score is at least 0.5.

The LLM judge evaluates the query, final answer, and an abbreviated tool trace against each scenario-specific rubric. This complements the executable checks by assessing requirements such as completeness and ordering, although the abbreviated trace limits verification of individual actions. Appendix P provides the judge prompt, input format, and score-handling rules.

The LLM judge evaluates the query, final answer, and an abbreviated tool trace against

scenario-specific rubrics. Together with executable checks, this provides complementary evidence about whether the agent satisfies the requested workflow, including completeness and ordering requirements. Appendix P provides the judge prompt and scoring details, while Appendix A.7 documents assertion semantics and validation limitations.

4 Experimental Setup

We evaluate eight LM configurations under a shared ReAct-style tool-calling protocol (Yao et al., 2023): Claude Sonnet 4.5, Claude Opus 4.1, o3, GPT-5, GPT-5.2, GPT-5.4, GPT-5.4-reasoning, and GPT-5.5-reasoning (Anthropic, 2025b,a; OpenAI, 2025b,a,c, 2026a,b). Each configuration is evaluated once on all 206 scenarios, using the same agent loop, a maximum of 15 iterations, and a prompt without demonstrations. Agent requests allow up to 4,096 completion tokens. Configuration names follow recorded deployment labels, whose mapping to public model snapshots is not independently verified; Appendix C.1 documents the identifiers and request settings.

All configurations receive the same 46 JSON-schema tools and are evaluated on a fixed synthetic corpus centered on the “*Vince Kaminski*” persona. The evaluated tool surface is a subset of the broader API specification and excludes boards, directory, and contact-group operations. Appendix N lists the exposed tools, while Appendix C.1 documents tool-access and persona restrictions. To examine how these restrictions affect the reported results, Section 5 presents the primary 206-scenario full-suite performance and a post hoc access-screened sensitivity analysis.

Outcomes are scored using the hybrid protocol in Section 3.3. Static checks inspect the evaluation snapshot and final answer, while the LLM judge assesses scenario-specific rubrics using the answer and abbreviated tool trace. Judge requests allow up to 2,048 completion tokens. The judge shares the endpoint used by the configuration labeled GPT-5, leaving possible self-evaluation bias unmeasured. Appendix P provides the prompt and configuration details.

Two additional configurations, labeled Phi-4 and Mistral Large, produced no tool invocations and achieved 0% pass rates in this setup. We report these additional runs in Appendix C.1, but not as part of the eight-configuration comparisons.

5 Results

5.1 Overall Performance

Table 3 reports the complete eight-configuration baseline. Claude Sonnet 4.5 has the highest observed pass rate on the primary 206-scenario full suite at 33.5%, followed by Claude Opus 4.1 at 29.6%. Rates among the evaluated GPT-5 variants range from 20.9% to 28.2%, with no monotonic ordering across all variants. We also report a post hoc access-screened sensitivity subset of 152 of the 206 scenarios. The screen retains a scenario only when every declared domain is exposed by the evaluated adapter, any explicit `user_id` identifies the Kaminski persona used by the fixed corpus, and the prompt does not require a known undeclared unavailable contact-group or directory operation. The three exclusion labels affect 19, 30, and 6 scenarios, respectively; one scenario has overlapping labels, producing a 54-scenario union. The six undeclared-operation cases are OB-H338, OB-X408, OB-X431, OB-X446, OB-X447, and OB-N573. Retained-subset rates are lower than removed-slice rates for every configuration (Table 3). The removed slice also contains proportionally more LLM-only scenarios (42.6% versus 13.2%) and fewer difficulty-1 scenarios (1/54 versus 16/152), showing that screening changes task composition rather than yielding a capability-valid subset. This descriptive post hoc screen does not repair grading, corpus, or other metadata limitations. All comparisons are descriptive because each configuration has one run.

Beyond pass rates, Table 3 reports static and LLM component scores to distinguish performance on executable checks from rubric-based assessments. These means cover different scenario sets: static scores average the 163 scenarios with static checks, whereas LLM scores average all 206. They therefore describe complementary aspects of performance and cannot simply be averaged to recover the mean scenario score. Appendix C.2 provides the full score breakdown.

The observed ranking summarizes one run per configuration, rather than establishing differences in repeated-run reliability. Figure 5 in the appendix supplements the point estimates with 95% Wilson intervals under a scenario-sampling interpretation. These intervals do not capture model or judge variability; their assumptions and interpretation are detailed in Appendix C.2.

Configuration	Pass	Full	Ret.	Rem.	Static	LLM	Calls	Call ok	Clean	Latency
Sonnet 4.5	69/206	33.5	28.3	48.1	.309	.419	5.68	99.7	94.7	27.06
Opus 4.1	61/206	29.6	25.0	42.6	.306	.419	5.37	99.7	96.6	40.56
GPT-5.4-R	58/206	28.2	24.3	38.9	.230	.372	3.61	96.8	88.8	21.84
GPT-5.5-R	57/206	27.7	25.0	35.2	.300	.407	8.65	94.1	88.3	31.69
GPT-5.4	53/206	25.7	20.4	40.7	.245	.333	2.66	95.4	89.3	17.80
GPT-5.2	47/206	22.8	18.4	35.2	.197	.286	1.86	92.4	58.7	15.84
o3	46/206	22.3	19.7	29.6	.214	.318	6.51	95.8	79.1	67.17
GPT-5	43/206	20.9	18.4	27.8	.204	.277	1.83	92.1	57.8	13.59

Table 3: Baseline results from one run per configuration. Pass and Full report the primary 206-scenario suite. Ret. and Rem. are pass percentages on the 152-scenario retained post hoc sensitivity subset and 54-scenario removed slice, respectively. Full, Ret., Rem., call success (Call ok), and clean-execution rate (Clean) are percentages. Static averages the 163 scenarios with static checks, while LLM averages all 206. Calls and latency in seconds are per-scenario means. Clean counts scenarios with at least one call and no observed API failure, using all 206 scenarios as the denominator.

Category	<i>n</i>	Son.	5.4-R	GPT-5
filters	10	80.0	40.0	20.0
folders	9	88.9	88.9	44.4
messages-write	15	73.3	46.7	53.3
messages-read	39	35.9	38.5	23.1
calendar-read	12	8.3	25.0	16.7
calendar-write	12	16.7	8.3	8.3
orchestration	17	17.6	11.8	11.8

Table 4: Pass rates for representative task categories. The complete eight-configuration table appears in Table 33.

5.2 Category and Composition Effects

Aggregate pass rates conceal substantial variation across task categories (Table 4). Sonnet 4.5 passes 88.9% of folder tasks (8/9), compared with 16.7% of calendar-write tasks (2/12), a 72.2-percentage-point difference. Its strong performance on filters (80.0%) and message writes (73.3%) also contrasts with its low orchestration pass rate (17.6%, 3/17). These results show that success on some state-changing tasks does not extend uniformly to other write operations or coordinated workflows; a simple read-versus-write distinction misses this variation. Within the evaluated task mix, these contrasts show that success on some state-changing tasks does not extend uniformly to other write operations or coordinated workflows. A simple read-versus-write distinction therefore misses substantial variation in performance.

Beyond task category, performance also varies with the number of API domains specified in each scenario’s metadata (Table 5). For example, a scenario tagged with both messages and calendar has two declared domains, regardless of the agent’s actual tool use. Sonnet’s pass rate declines from

Domains	Son.	Opus	5.4-R	o3
1 (<i>n</i> = 122)	37.7	33.6	34.4	27.9
2 (<i>n</i> = 53)	34.0	28.3	26.4	20.8
3 (<i>n</i> = 24)	16.7	16.7	8.3	4.2
4 (<i>n</i> = 7)	14.3	14.3	0.0	0.0

Table 5: Pass rates by declared domain count. Cohorts differ in task composition and tool availability.

37.7% on one-domain tasks to 34.0%, 16.7%, and 14.3% on tasks declaring two, three, and four domains, respectively. GPT-5.4-reasoning shows a similar pattern, with corresponding rates of 34.4%, 26.4%, 8.3%, and 0%. These results associate broader domain coverage with lower task success, although the cohorts differ in task composition and contain only seven four-domain scenarios.

Unlike domain count, author-assigned difficulty does not show a monotonic relationship with pass rates. The three tiers designate basic (D1), intermediate (D2), and complex (D3) tasks, but both Sonnet and GPT-5.4-reasoning perform best on D2 and worst on D1 (Table 19). Sonnet passes 11.8%, 42.1%, and 23.5% of D1, D2, and D3 tasks, respectively; GPT-5.4-reasoning passes 5.9%, 37.2%, and 17.6%. The tiers contain 17, 121, and 68 scenarios, respectively. The present analysis does not establish why D1 tasks have lower pass rates than D2; explaining this pattern requires examining task composition and grading outcomes within each tier.

5.3 Execution is not task completion

To examine whether successful tool execution translates into task completion, we compare API-level call outcomes with scenario-level grades. Sonnet issues 1,170 calls, of which 99.7% complete without an observed API failure. Opus also reaches

Scenario	Required workflow and available checks
OB-H312	Aggregate unread percentages by sender; retain rates above 50% and sort descending. Two static assertions and one rubric check the answer, with a known literal-pattern limitation.
OB-N520	Create Projects with Raptor, Dynegy, and FERC subfolders, then route keyword matches using returned IDs. Five static assertions and one rubric check the hierarchy and routing.
OB-H307	Route automated, high-importance, then remaining unread messages in order; exclude Sent Items. Six static assertions and one rubric do not provide equivalent coverage of the full workflow.

Table 6: Concrete task and grading requirements. Complete queries and assertions appear in Appendix L. These cases illustrate coverage differences rather than a sampled error taxonomy.

99.7%, while the remaining configurations range from 92.1% to 96.8%. Yet scenario pass rates remain between 20.9% and 33.5%. This contrast highlights the distinction between an operation executing successfully and satisfying the user’s request: a call can retrieve the wrong entities, apply an incomplete filter, or perform only part of a required workflow.

Recorded trajectories illustrate how this gap arises in practice. In a successful folder-routing scenario, the agent creates a parent folder and three children, searches by project keyword, and uses the returned folder identifiers to route messages. In an unsuccessful inbox-triage scenario, all six static checks pass, but the agent reaches the iteration limit before completing the ordered workflow or reporting final counts. The critical LLM rubric fails, producing a failing outcome despite an aggregate score of 0.50. Together, these examples illustrate why neither successful API execution nor satisfaction of individual static checks alone establishes complete workflow execution.

The recorded runs for the tasks in Table 6 contrast completed routing with continued activity that does not finish the requested workflow. In OB-N520, Sonnet makes ten calls over five iterations: one parent-folder creation, three child-folder creations, three searches, and three batch moves, followed by a final report. Both the static and LLM components score 1.0. In contrast, OB-H307 uses 29 calls across the full 15 iterations without completing the requested triage. After initial routing, the agent resumes message listing, creates a second set of the three named folders, and continues

retrieval until the limit. Its final answer announces further retrieval rather than reporting completion. This case illustrates that additional tool activity need not yield task completion, but does not isolate agent planning from environment constraints such as frozen message reads and folder counters that do not reflect mutations. Appendix Q provides the iteration-level evidence and grading rationale.

The reported execution rates measure whether calls complete without an observed API failure, not whether they perform the intended action or satisfy the task. We classify these outcomes from structured API responses; Appendix C.7 details the classification rules, handling of truncated results, and error-continuation statistics. Tool engagement, answer production, and task completion must also be distinguished: an agent may answer after an API error, while a recorded pass need not involve a tool call. Appendix C.6 and Appendix C.5.6 report these indicators and the score distributions. We next examine engagement and completion timing to distinguish responses without tool calls from attempted but unfinished workflows.

5.4 Engagement and Completion Timing

Low task success can coincide with either limited tool engagement or unsuccessful execution after tools are invoked. To distinguish these patterns, Table 7 separates failures with and without recorded calls and reports when passing trajectories finish. GPT-5 and GPT-5.2 each have 71 zero-tool failures (34.5% of all scenarios), compared with seven for Sonnet and four for Opus. This contrast helps explain why mean call counts alone are insufficient to characterize agent behavior: low averages may reflect responses without tool use rather than efficient workflow execution. However, the absence of calls identifies behavior, not intent; distinguishing refusals, reports of missing capabilities, and unsupported answers requires inspecting the responses.

Among trajectories that eventually pass, completion timing shows how success is distributed across the interaction budget. Figure 2 counts recorded passing trajectories that finish by iteration K under the original 15-iteration budget. By iteration five, Sonnet has completed 62 of its 69 passes (89.9%), compared with 30 of 46 for o3 (65.2%) and 40 of 43 for GPT-5 (93.0%). Late completions therefore account for a larger share of o3’s recorded successes than those of Sonnet or GPT-5. These retrospective curves do not estimate performance under a reduced budget, which could alter agent

Configuration	Fail/calls	Fail/none	Pass by 5
Sonnet 4.5	130	7	62/69
Opus 4.1	141	4	45/61
GPT-5.4-R	138	10	47/58
GPT-5.5-R	144	5	38/57
GPT-5.4	140	13	48/53
GPT-5.2	88	71	42/47
o3	136	24	30/46
GPT-5	92	71	40/43

Table 7: Failures with and without recorded calls, and eventual passes completed within five iterations. Each row’s two failure counts plus its total passes sum to 206. The final column conditions on the original run’s passing trajectories; it is not a five-iteration rerun.

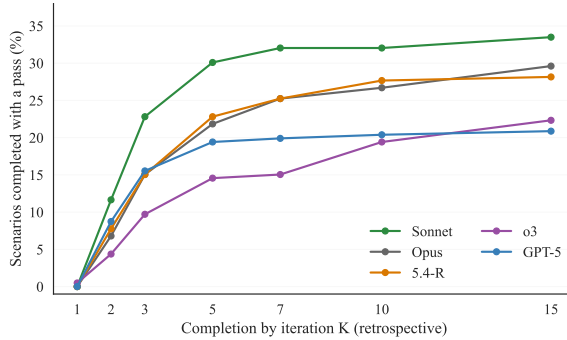


Figure 2: Retrospective completion of passing trajectories. Values use all 206 scenarios as the denominator. By iteration five, 65–93% of each configuration’s eventual passes have finished. These curves do not estimate performance under a reduced iteration budget.

behavior and final answers. Appendix C.5 provides the complete thresholds and activity profiles.

Tool-call counts provide a complementary view of activity in successful and unsuccessful runs. Sonnet averages 5.28 calls on passing scenarios and 5.88 on failures; o3 averages 4.63 and 7.06, respectively. GPT-5 shows the reverse pattern, with 3.02 calls on passes and 1.52 on failures, partly because its many zero-tool failures lower the failure-group mean. Thus, greater tool activity does not consistently distinguish passing from failing trajectories. Because these groups contain different tasks, the comparisons do not establish whether additional calls improve task success. Appendix C.5.4 reports the full outcome-conditioned statistics, while Appendix C.5.3 separately examines repeated-call patterns.

5.5 Shared & Configuration-Specific Outcomes

Beyond differences in tool use and completion timing, we examine whether configurations succeed

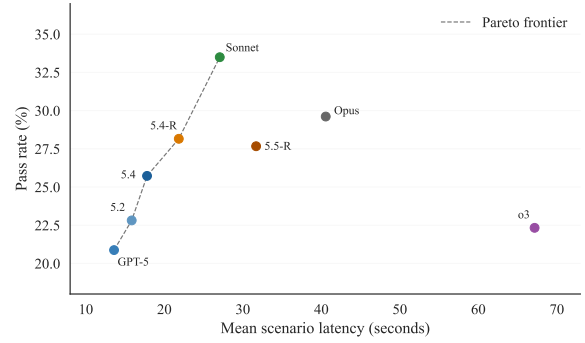


Figure 3: Observed tradeoff between mean scenario latency and pass rate among the evaluated configurations. Sonnet achieves the highest pass rate, while several GPT-5 configurations offer shorter mean latency at lower task success.

on the same scenarios. For the selected subset of Sonnet, Opus, GPT-5.4-reasoning, and o3, 24 of 206 scenarios pass for all four, 117 pass for none, and 65 have differing outcomes. Together, their recorded pass sets cover 89 scenarios (43.2%), including 20 beyond Sonnet’s 69 passes. This broader coverage indicates complementary successes within the subset, but is not an evaluated ensemble or routing result.

Pairwise comparisons show that this overlap varies across configurations. Jaccard similarity, defined as the intersection of two pass sets divided by their union, is 0.711 for Sonnet and Opus, compared with 0.530 for Sonnet and GPT-5.4-reasoning and 0.420 for Sonnet and o3. Thus, Sonnet and Opus share a larger proportion of their successful scenarios than either of these other pairings. Table 21 reports all six comparisons within the selected subset, which is not the top four configurations by pass rate.

The 117 scenarios failed by all four provide a complementary view of shared weaknesses. Calendar-domain tasks account for 45.3% of this set, compared with 34.5% of the full suite. Tasks declaring multiple domains account for 46.2% versus 40.8%, and D3 tasks account for 40.2% versus 33.0%. Calendar, multi-domain, and D3 tasks are therefore overrepresented among these shared failures. These overlapping patterns identify candidates for closer inspection, but do not separate agent limitations from tool-access, persona, or grading constraints. Appendix S and Table 39 provide the complete cohort profile.

5.6 Latency and Task Success

Finally, we examine task success alongside response time to characterize the practical tradeoffs among configurations. Sonnet achieves the highest observed pass rate, 33.5%, at a mean latency of 27.06 seconds per scenario (Figure 3). GPT-5.4-reasoning reaches 28.2% at 21.84 seconds, while GPT-5 reaches 20.9% at 13.59 seconds. Sonnet is both more successful and faster on average than Opus and o3, but several GPT-5 configurations finish sooner at lower pass rates. The preferred configuration therefore depends on the relative importance of task success and response time.

Recorded latencies include hosted-service effects and task-dependent trajectory lengths, so they characterize the evaluated agent systems rather than isolated model-inference speed. Table 30 provides the complete latency summary, and Appendix D.4 discusses alternative efficiency measures.

6 Discussion

The central finding is a gap between executing operations and completing the user’s workflow. For Sonnet, 94.7% of scenarios contain at least one tool call and no observed API failure, yet only 33.5% pass the task-level grading protocol. The inbox-triage example illustrates a related distinction: all six static checks pass, but the critical completeness rubric fails. These results show that successful calls and satisfied individual checks can coexist with an unfinished task. In enterprise email workflows, the relevant outcome includes selecting the intended entities, preserving dependencies between actions, and completing the requested sequence.

This distinction motivates evaluating agent behavior at multiple levels without treating those levels as interchangeable. Call-level checks identify execution failures, while targeted state and answer checks verify specified requirements. Workflow-level assessment must additionally consider whether those requirements collectively cover the user’s request. EmailBench combines these forms of evidence through its hybrid protocol, but the combination does not itself establish agreement with human judgment. Expert calibration and targeted grader comparisons are needed to assess whether the implemented checks adequately capture task completion.

The findings also motivate interventions that track progress toward the requested outcome, rather than merely increasing tool activity. Explicit depen-

dency tracking could preserve returned identifiers and action ordering, while completion checks could identify unmet requirements before an agent stops. These are hypotheses for future experiments, not mechanisms validated by the present results. Such experiments should provide task-appropriate tools and persona data, ensure that subsequent reads reflect the intended state semantics, and use repeated runs. Empirical calibration of difficulty labels would also help distinguish anticipated complexity from observed performance.

6.1 Reproducibility and Availability

The artifact includes the benchmark implementation, synthetic seed corpus, scenario definitions, executable assertions, LLM rubrics, and evaluation adapter. The TypeScript environment operates without a live email provider, while model and judge inference requires external services and authorized credentials. Reported aggregates are derived from recorded outcomes joined to scenario identifiers, supporting inspection of both results and their underlying traces. The records retain endpoint identifiers and timestamps, but not a complete source-commit and tool-schema manifest. Appendix G and Appendix U document the available artifacts, execution requirements, and provenance limits. A repository-wide license covering all artifacts has not been identified, so unrestricted reuse or redistribution should not be assumed.

7 Conclusion

We presented EmailBench, a self-contained benchmark for enterprise email and productivity agents. Across eight configurations in the evaluated single-user setting, the highest observed task pass rate is 33.5%, despite high API-level execution success. Category-level results and recorded trajectories demonstrate that executing individual operations does not establish completion of the requested workflow. EmailBench provides a concrete setting for studying this gap and developing evaluations that connect local execution evidence to end-to-end email-task outcomes. These findings motivate a task-centered standard for enterprise email agents: success means completing the user’s requested workflow, not merely executing its individual operations.

Limitations

The simulated environment isolates email workflows from operational conditions such as network failures, rate limits, eventual consistency, and authentication. Its 65-message corpus supports controlled evaluation but does not reproduce the retrieval and pagination demands of production mailboxes. All content and queries are in English, and tasks involve a single user request rather than interactive clarification or correction.

The evaluated setup does not support every scenario requirement. All runs use the same persona corpus, including 30 scenarios specifying another user, and the supplied tools omit board, directory, and contact-group operations. Message collection reads remain tied to the seed corpus rather than consistently reflecting recorded mutations, which can affect workflows that verify earlier actions through subsequent reads. Calendar construction also truncates fractional hours, creating discrepancies with some duration-based rubrics. Outcomes on affected tasks therefore combine agent behavior with tool-access, corpus, and environment constraints. The access-screened analysis partially controls for known access and persona mismatches, but remains descriptive and does not resolve other environment or grading issues.

The hybrid grader provides complementary evidence, not an exhaustive or independently validated measure of task correctness. Static checks verify selected conditions, and 43 scenarios rely entirely on LLM rubrics. The judge receives abbreviated tool traces, while some answer checks use literal substring matching despite regex-like patterns. These choices can limit verification or penalize answers for reasons other than task correctness. We report no independent expert calibration or alternate-judge evaluation. The judge also shares an endpoint with the configuration labeled GPT-5, leaving possible self-evaluation bias unmeasured.

Each configuration has one recorded run per scenario, so the study does not estimate variability across repeated executions or judgments. Configuration names follow recorded deployment labels whose mapping to public model snapshots is not independently verified. Category, domain-count, and difficulty comparisons involve different task compositions and sometimes small cohorts; they do not isolate causal effects. Likewise, selected trajectories illustrate behaviors rather than establish their prevalence. The reported results characterize

the evaluated agent, environment, and grading protocol together, rather than isolating model behavior from access and grading constraints.

Ethical Considerations

The benchmark uses synthetic content inspired by themes and publicly documented organizational relationships in the Enron collection (Klimt and Yang, 2004). The checked-in corpus contains no original email bodies or private mailbox exports but retains names associated with real people, person-linked email addresses, roles, affiliations, and contact fields. Synthetic content should therefore not be equated with anonymized data, and generated messages or actions should not be interpreted as statements or conduct of the named individuals. Public availability of the source collection does not establish unrestricted rights to reuse or redistribute every underlying item or identifier. The available artifacts contain aggregate prototype intent-frequency counts but do not establish how the underlying telemetry was collected or de-identified; no stronger privacy assurance is supported.

The benchmark executes actions in a simulated environment, but analogous actions in live systems can send messages, move records, or schedule meetings with real consequences. Its results support studying reliability and evaluation methods, not certifying models for unsupervised deployment. Adaptation to live systems requires appropriate authorization, confirmation of consequential actions, audit logging, and recovery mechanisms. Evaluation should also consider whether an agent respects these safeguards, rather than rewarding task completion regardless of how it is achieved.

References

- Anthropic. 2025a. Claude Opus 4.1. <https://www.anthropic.com/news/claude-opus-4-1>. Accessed: 2026-09-18.
- Anthropic. 2025b. Claude Sonnet 4.5 System Card. <https://www.anthropic.com/system-cards>. Accessed: 2026-09-18.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.
- Bryan Klimt and Yiming Yang. 2004. *The Enron corpus: A new dataset for email classification research*. In *Machine Learning: ECML 2004*, pages 217–226. Springer.
- Xiangyi Li, Kyoung Whan Choe, Yimin Liu, Xiaokun Chen, Chujun Tao, Bingran You, Wenbo Chen, Zonglin Di, Jiankai Sun, Shenghan Zheng, Jiajun Bao, Yuanli Wang, Weixiang Yan, Yiyuan Li, and Han-chung Lee. 2026. *ClawsBench: Evaluating capability and safety of LLM productivity agents in simulated workspaces*. *arXiv preprint arXiv:2604.05172*.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. *AgentBench: Evaluating LLMs as agents*. In *International Conference on Learning Representations*.
- OpenAI. 2025a. GPT-5 system card. <https://cdn.openai.com/gpt-5-system-card.pdf>. Accessed: 2026-09-18.
- OpenAI. 2025b. OpenAI o3 and o4-mini system card. <https://openai.com/index/o3-o4-mini-system-card/>. Accessed: 2026-09-18.
- OpenAI. 2025c. Update to the GPT-5 system card: GPT-5.2. <https://openai.com/index/gpt-5-system-card-update-gpt-5-2/>. Accessed: 2026-09-18.
- OpenAI. 2026a. GPT-5.4 Thinking system card. <https://openai.com/index/gpt-5-4-thinking-system-card/>. Accessed: 2026-09-18.
- OpenAI. 2026b. GPT-5.5 system card. <https://openai.com/index/gpt-5-5-system-card/>. Accessed: 2026-09-18.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. *Gorilla: Large language model connected with massive APIs*. In *Advances in Neural Information Processing Systems*, volume 37.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. *ToolLLM: Facilitating large language models to master 16000+ real-world APIs*. In *International Conference on Learning Representations*.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. *AppWorld: A controllable world of apps and people for benchmarking interactive coding agents*. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076. Association for Computational Linguistics.
- Gust Verbruggen, Chris Parnin, Vu Le, and Sumit Gulwani. 2026. *JQBench: A benchmark for reading and editing JSON from natural language and/or examples*. In *International Conference on Learning Representations*.
- Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. 2025. *Can LLMs help you at work? a sandbox for evaluating LLM agents in enterprise environments*. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9167–9201. Association for Computational Linguistics.
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025. *OdysseyBench: Evaluating LLM agents on long-horizon complex office application workflows*. *arXiv preprint arXiv:2508.09124*.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024. *OfficeBench: Benchmarking language agents across multiple applications for office automation*. *arXiv preprint arXiv:2407.19056*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. *OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments*. In *Advances in Neural Information Processing Systems*, volume 37.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. *ReAct: Synergizing reasoning and acting in language models*. In *International Conference on Learning Representations*.
- Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. *AssistantBench: Can web agents solve realistic and time-consuming tasks?* In *Proceedings of the 2024 Conference on Empirical Methods*

in *Natural Language Processing*, pages 8938–8968. Association for Computational Linguistics.

Shiyue Zhang, Asli Celikyilmaz, Jianfeng Gao, and Mohit Bansal. 2021. [EmailSum: Abstractive email thread summarization](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6895–6909. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging LLM-as-a-judge with MT-Bench and Chatbot Arena](#). *Advances in Neural Information Processing Systems*, 36.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [WebArena: A realistic web environment for building autonomous agents](#). In *International Conference on Learning Representations*.

Domain	R	W	Key Ops
messages	7	15	search, send, reply, flag, batch
calendar	7	11	create, RSVP, schedule, re-minders
boards	6	9	columns, tasks, assignment
settings	6	9	vacation, labels, working hrs
todo	4	10	lists, tasks, check-lists
folders	5	5	create, rename, move, tree
contacts	5	5	create, folders, search
filters	2	3	create, update, delete
directory	5	0	search, manager, reports
contactGroups	3	1	members, add member
identity	1	0	current user
Total	51	68	119 methods

Table 8: API domain coverage showing read (R) and write (W) method counts. The 51/68 split excludes the two evaluation utilities. The evaluated adapter is reported separately.

A Supplementary Benchmark Design

This appendix documents the API inventory, corpus construction, scenario metadata, and grading implementation. It also provides the complete evaluated tool surface, prompts, examples, and supplementary analyses needed to interpret the main-paper results.

A.1 Provider-Neutral API Specification

The EmailBenchAPI interface defines 119 operational methods across ten domain objects plus identity: 51 reads and 68 writes. Two evaluation utilities, `reset()` and `snapshot()`, bring the complete interface to 121 methods. The operational inventory differs from the evaluated 46-tool adapter. Table 8 summarizes the coverage, breaking down read and write operations per domain.

Design Principles. Three principles guide the API design:

(1) Provider neutrality. All types use generic terminology, such as `MailFilter` rather than `InboxRule`, `VacationResponder` rather than `AutomaticReplies`, and `Label` rather than `MasterCategory`. A provider-specific implementation can map these contracts to a concrete service.

(2) Typed contracts. Every method has fully typed input and output signatures using 36 typed interfaces (`Message`, `Event`, `Contact`, `ToDoTask`, `BoardTask`, `MailFolder`, `MailFilter`, etc.). Typed contracts prevent ambiguity in both agent code generation and programmatic grading.

(3) Observable state. The `snapshot()` method returns an `APISnapshot`. It contains selected entity collections and three mutation logs (`sentMessages`, `deletedIds`, and `movedMessages`), which support deterministic post-hoc grading without temporal coupling to agent execution.

Implementations. We provide two implementations. **MockOutlookAPI** (655 LOC) is an in-memory mutable API with OData-style filtering and mutation tracking. **ChangelogEmailBenchAPI** (1,034 LOC) reads from a frozen corpus and captures writes as typed changelog entries with 66 entry types, providing immutable provenance for grading.

A.2 Corpus Construction and Adaptation

EmailBench’s seed corpus draws on the publicly available Enron email dataset (Klimt and Yang, 2004) through the following multi-stage process.

Source Material. The Enron corpus contains ~500,000 emails from 150 employees, released by the Federal Energy Regulatory Commission (FERC) during the Enron investigation. We use Vince Kaminski, publicly identified as VP of Research, as the primary persona. The synthetic content covers energy trading, risk modeling, and regulatory compliance across messages, meetings, contacts, and tasks. No original or private email body is included, although public names, roles, addresses, and organizational relationships remain.

Adaptation Pipeline. The adaptation proceeds in four stages:

Stage 1: Theme extraction. We analyze the raw Enron maildir to extract recurring themes (energy trading, FERC filings, VaR models, Project Raptor, Dynegy partnership), organizational relationships (Kaminski → Skilling, Beck, Kitchen, Lay), and communication patterns (executive summaries, meeting follow-ups, regulatory alerts).

Stage 2: Synthetic message generation. Rather than directly importing Enron emails, we *synthesize* 65 new messages whose bodies and sub-

Entity	N	Entity	N
Messages	65	Boards	2
Events	15	Board columns	6
Contacts	15	Board tasks	7
Folders	6	Users	16
Filters	1	Labels	6
Calendars	3	Contact groups	3
Todo lists	3	Contact folders	3
Todo tasks	10	Attachments	11

Table 9: Seed corpus entity counts. Actual message membership is 50 in Inbox and 15 in Sent Items. No message has Archive as its parent folder.

Connection	Description
Email ↔ Calendar	Meeting attendees (Skilling, Beck, Kitchen) also appear as email senders, and threads reference upcoming meetings
Email ↔ Contacts	All major senders have contact records with titles, departments, company
Calendar ↔ Tasks	Todo items reference projects discussed in events ("Raptor" tasks, "FERC" prep)
Conflicts	Two events (FERC Review, Dynegy Review) deliberate overlap for conflict detection
Mixed signals	System accounts (EnronOnline, GasBank) generate noise requiring differential handling

Table 10: Cross-entity interconnections enabling multi-domain scenarios.

jects are fabricated while retaining some publicly documented names, address strings, roles, and organizational relationships. Messages are generated via a deterministic factory function with controlled parameters: sender, subject template, temporal offset (`hoursAgo/daysAgo` relative to the reference time 2001-10-15T10:00:00Z), read state (`isRead = msgSeq % 3 ≠ 0`), importance level, and conversation threading via shared `conversationId` values.

Stage 3: Productivity entity enrichment. Since the source corpus contains only emails, we add synthetic calendar events (15), contacts (15), todo tasks (10), board tasks (7), one mail filter, and 15 organizational directory entries. These entity types support productivity workflows that are absent from the source data. Each synthetic entity is grounded in the message themes: calendar events reference meetings mentioned in emails, contact records correspond to email senders, and todo tasks relate to projects discussed in message threads.

Stage 4: Cross-entity interconnection. We deliberately introduce cross-references to enable realistic multi-domain scenarios. Table 9 summarizes the final entity counts and Table 10 details the interconnection types.

Source selection. The Enron collection provides a publicly released enterprise setting with documented organizational relationships and multi-topic business communication. EmailBench uses these public themes and relationships as scaffolding for fabricated content. SpamAssassin, holiday-calendar, and conference-schedule data are included as resources for possible extensions but are not part of the reported corpus.

A.3 Benchmark Construction Methodology

EmailBench’s 206 scenarios were developed in four batches. Aggregate intent-frequency counts from an Outlook-agent prototype informed the task inventory.

Telemetry-Informed Design. Aggregate intent-frequency counts from an Outlook-agent prototype were used to identify common user intents and API call patterns. The available artifacts do not establish how the source telemetry was collected or de-identified. The top-10 intents by frequency are: (1) search and summarize messages, (2) schedule meetings, (3) forward/reply to threads, (4) organize inbox into folders, (5) create and assign tasks, (6) manage calendar conflicts, (7) set up mail filters, (8) update vacation settings, (9) prepare meeting briefs, and (10) triage and prioritize incoming mail. Each of the 16 benchmark categories maps to one or more of these observed intent classes. The aggregate counts informed topic selection but do not establish that the benchmark distribution matches production use.

Four-Batch Construction. The scenarios were constructed in four iterative batches: **Batch 1** (40 tasks): multi-step operations identified as failure modes during prototype testing. **Batch 2** (50 tasks): coverage-gap filling for under-represented domains (calendar writes, contacts, boards). **Batch 3** (32 tasks): tasks with multiple named personas. The evaluated runner does not switch corpus identity for these tasks. **Batch 4** (84 tasks): systematic gap-filling to ensure ≥ 8 scenarios per category (details in Appendix I).

Available validation evidence. The released artifacts include scenario definitions, assertion implementations, and recorded assertion outcomes. They do not include per-scenario review records that would support a claim of independent manual validation. The adapter omissions and assertion limitations described below therefore remain part

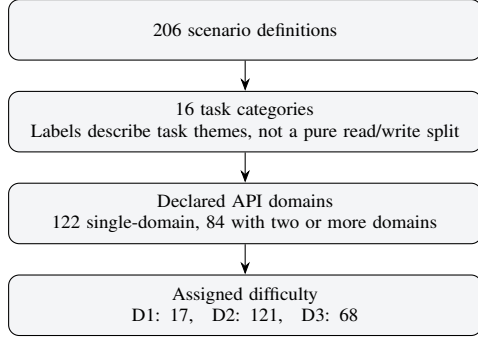


Figure 4: Scenario metadata axes. Category, domain count, and difficulty are separate labels. D3 does not require three or more domains.

Tier	Definition	N	%
D1	Basic assigned tasks	17	8
D2	Intermediate assigned tasks	121	59
D3	Complex assigned tasks	68	33

Table 11: Assigned difficulty tiers from scenario definitions. Overall, 92% are D2 or D3. Actual tool-call and domain counts need not follow a fixed threshold.

of the evaluation protocol.

A.4 Task Taxonomy

We organize the 206 scenarios along three metadata axes (Figure 4): *capability* (what the task tests), *domain* (which API domains are declared), and *assigned difficulty* (the scenario author’s qualitative complexity label).

Capability Axis. Tasks retrieve and analyze data, create and mutate entities, or combine these actions into workflows. We do not report an aggregate read-/write partition without an explicit scenario-level classification. For example, contacts includes mutations, and folders, filters, and settings frequently require writes.

Domain Axis. Each scenario is tagged with the API domains it exercises. The median declared domain count is 1 overall and 2 for D3 scenarios. The most common co-occurring pairs are messages+calendar (42/206), messages+todo (25/206), and calendar+todo (18/206). A scenario may contribute to several pairs.

Difficulty Axis. Table 11 formalizes the three difficulty tiers.

A.5 Scenario Design

EmailBench comprises 206 scenarios across 16 categories (Table 12).

Category	N	Category	N
messages-read	39	todo	9
orchestration	17	meeting-prep	9
cross-domain	15	inbox-cleanup	9
messages-write	15	folders	9
multi-domain	12	calendar-triage	8
calendar-read	12	settings	8
calendar-write	12		
contacts	12		
boards	10		
filters	10		

Table 12: Scenario category distribution (206 total).

Category Design Rationale. The categories describe task themes. The following groups are descriptive and may overlap:

Read tasks (messages-read, calendar-read, contacts) test retrieval, filtering, aggregation, and analysis.

Write tasks (messages-write, calendar-write, folders, filters, settings) test state mutations. Difficulty varies within writes: folder, filter, and settings tasks perform state changes yet achieve higher pass rates than calendar writes. Category labels alone do not define a controlled read-versus-write comparison.

Cross-domain tasks (orchestration, cross-domain-analytics, multi-domain-workflow, meeting-prep, inbox-cleanup, calendar-triage) test whether the agent can correlate data across API domains and execute multi-step pipelines.

Representative Scenario Examples. Table 13 illustrates three workflows for which recorded evaluation traces are available.

A.6 Hybrid Grading Pipeline

EmailBench evaluates agent performance through two complementary assertion modes, combined into a single score.

A.6.1 Static Assertions

The assertion interface supports nine types (Table 14). The 206 evaluated scenarios instantiate 258 static assertions using four types. Snapshot checks and answer-substring checks have different inputs.

A.6.2 LLM-Judge Assertions

For aspects that resist programmatic checking (e.g., “Generate a professional meeting prep brief”), we use an LLM judge (Zheng et al., 2023) that scores

ID	Evaluated query and grading
OB-H312	D2, messages-read. For each person who emailed me, calculate what percentage of their emails to me are unread. Show only people with >50% unread rate, sorted by unread percentage descending. 2 static assertions and 1 LLM rubric.
OB-N520	D3, folders. Create a folder hierarchy: 'Projects' with subfolders 'Raptor', 'Dynegy', and 'FERC'. Then move all emails mentioning each project keyword to the appropriate subfolder. 5 static assertions and 1 LLM rubric.
OB-H307	D3, inbox-cleanup. Set up an email triage system: (1) Create folders 'Action Required', 'FYI Only', and 'Automated'. (2) Move all emails from automated senders (enrononline, legal-tickets, gasbank) to 'Automated'. (3) Move all high-importance emails to 'Action Required'. (4) Move all remaining unread emails to 'FYI Only'. Do NOT move any emails that are already in Sent Items. 6 static assertions and 1 LLM rubric.

Table 13: Three evaluated scenario definitions with recorded traces. OB-H312 analyzes senders, OB-N520 creates a hierarchy, and OB-H307 performs triage. Appendix L details their checks.

Type	What it checks
count	Collection size matches expected value (eq, gte, lte, gt, lt operators)
exists	Predicate returns true on snapshot
not-exists	Predicate returns false on snapshot
property	Strict equality on a computed value
sent-message	Sent message matches subject/to/body
moved-message	Messages moved to target folder
deleted	Minimum entities deleted
state-check	Arbitrary boolean predicate on snapshot
return-contains	Agent answer contains pattern

Table 14: Nine supported static types. Snapshot checks inspect `APISnapshot`, while `return-contains` checks the final answer. Four types occur in the evaluated suite.

against rubrics across 7 semantic dimensions: *correctness*, *completeness*, *relevance*, *format*, *safety*, *action-taken*, and *no-hallucination*.

The judge receives the scenario query, the agent’s text answer, a truncated tool-call trace (arguments capped at 100 characters and results at 200 per call), and the rubric. It returns a JSON object containing a score in $[0, 1]$ and a one-sentence rationale. The evaluator clamps a parsed score to this range and derives the critical-check outcome by comparing it with the 0.5 threshold. An unparseable response or judge-call error produces a failing zero score. Run logs identify judge deployment `dev-gpt-5-chat-jj`; the checked-in request omits temperature.

A.6.3 Scoring

When both assertion components are present, the final score is

$$s = \frac{1}{2}s_{\text{static}} + \frac{1}{2}s_{\text{LLM}}.$$

When only one component is present, its score is used directly.

A scenario *passes* when $s \geq 0.5$ **and** all assertions marked `critical: true` pass. The critical flag enables hard requirements (e.g., “the folder must exist”) while allowing soft scores for quality aspects. The evaluated baseline uses equal component weights when both exist and the available component otherwise. These rules depend on assertion availability, not on whether a task reads or writes. Reported scores and pass flags are taken from the recorded runs.

A.7 Grading Inventory and Validation Limits

Across 206 scenarios, 258 static assertions give a mean of 1.252 per scenario. Forty-three scenarios have no static assertion. The suite contains 211 LLM assertions. Appendix J gives the static inventory.

Result files include assertion outcomes and LLM-judge rationales. The available artifacts do not include the expert annotations needed to estimate inter-rater agreement or conduct a grader ablation. We therefore make neither claim.

The executed `return-contains` checker uses literal substring matching, although some scenario patterns are written as regular expressions. For example, OB-H312’s percentage pattern is treated literally even when an answer includes a percentage. The reported grades follow these executed semantics. Changing them requires a separately reported regrading analysis. Recorded final answers and patterns permit offline reevaluation of this assertion class.

B Supplementary Benchmark Comparison

Table 15 situates EmailBench among general agent, office-workflow, and enterprise benchmarks. The columns report properties described by each benchmark and are not intended as quality rankings.

The comparison highlights differences in scope and evaluation abstraction. EmailBench centers on email and adjacent productivity domains, while EnterpriseBench covers cross-functional enterprise work, OdysseyBench emphasizes long-horizon office workflows, and ClawsBench evaluates stateful productivity services and safety. These design differences preclude direct comparison of task counts or headline scores.

Benchmark	Domain	Tasks	API	Typed	Grading	Offline	Multi-D	Corpus
AgentBench	8 envs	1,013	N/A	✗	Functional	✓	✗	Varied
WebArena	Web	812	N/A	✗	Match	✗	✗	Live sites
SWE-bench	Code	2,294	N/A	✗	Unit tests	✓	✗	GitHub
OfficeBench	Office apps	300	N/A	✗	Match/exec.	✗	✓	Synthetic
AppWorld	9 apps	750	457	✓	API state	✓	✓	Synthetic
EnterpriseBench (Vishwakarma et al., 2025)	Enterprise	500	N/A	N/A	Task-specific	✓	✓	Synthetic
OdysseyBench (Wang et al., 2025)	Office apps	602	N/A	N/A	Task-specific	N/A	✓	Mixed
ClawsBench (Li et al., 2026)	5 services	44	198	N/A	State/safety	✓	✓	Synthetic
Ours	Email+	206	119	✓	Hybrid	✓	✓	Adapted

Table 15: Comparison with related benchmarks. “API” denotes a reported method, endpoint, or route count. Counts are not directly equivalent across abstractions. “Multi-D” denotes cross-domain or cross-application scenarios. N/A indicates that the cited paper does not report a directly comparable value.

C Supplementary Experimental Results

C.1 Evaluation Configuration

We report eight recorded configurations under labels from three model groups using a standardized ReAct-style tool-calling agent (Yao et al., 2023): **OpenAI GPT-5 labels:** GPT-5, GPT-5.2, GPT-5.4, GPT-5.4-reasoning, and GPT-5.5-reasoning (OpenAI, 2025a,c, 2026a,b). **OpenAI o3 label:** o3 (OpenAI, 2025b). **Anthropic labels:** Claude Opus 4.1 and Claude Sonnet 4.5 (Anthropic, 2025a,b). The recorded files identify internal gateway endpoints rather than public model snapshots (Table 16). The saved o3 alias maps to dev-gpt-5-reasoning, so the underlying model family is not independently established. Phi-4 and Mistral Large produced no tool invocations in this configuration and received a 0% pass rate. They are disclosed here but excluded from the eight-configuration analyses below.

Agent Architecture. The evaluated adapter supplies 46 JSON-schema tools: messages (13), folders (6), filters (4), calendar (7), contacts (4), todo (6), settings (5), and identity (1). It constructs `ChangelogEmailBenchAPI(CORPUS)` and runs for up to 15 iterations. Board, directory, and contact-group tools are absent. No few-shot examples are supplied. Every scenario uses the fixed Kaminski corpus: the runner ignores `scenario.user_id`. Of 206 scenarios, 34 declare a user ID and 30 specify another user. These results describe this restricted setup, not genuine multi-user evaluation. Expanded tool access or user-specific corpora require a revised experiment and reruns. Each scenario is run once per model. The checked-in agent request omits temperature and sets `max_completion_tokens=4096`. The 180-second agent bridge limit applies per request, not per scenario. Judge re-

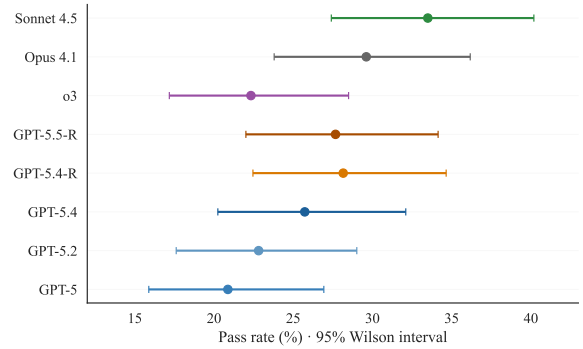


Figure 5: Recorded pass rates with two-sided 95% Wilson score intervals ($n = 206$ per model). These intervals describe hypothetical scenario sampling, not repeated-run or judge variability. Sonnet has the highest observed rate, 69/206 (33.5%).

quests use a 120-second subprocess limit and `max_completion_tokens=2048`; the bridge uses a 120-second HTTP timeout.

C.2 Component Scores and Intervals

Table 17 and Figure 5 present the main results. For these intervals, let $\hat{p} = k/n$, $n = 206$, and $z = 1.959964$. Wilson center and half-width are

$$c = \frac{\hat{p} + z^2/(2n)}{1 + z^2/n},$$

$$h = \frac{z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

We report $[c - h, c + h]$ without continuity correction. This binomial interpretation treats scenarios as independent samples. The fixed-suite rate is already known exactly, task dependence is not modeled, and these intervals do not establish pairwise model significance.

Claude Sonnet 4.5 has the highest observed pass rate at 33.5%, followed by Opus 4.1 at 29.6%. Across the reported GPT-5 configurations,

Configuration label	Internal endpoint ID	Run ID	Start time (UTC)
Claude Sonnet 4.5	dev-anthropic-claude-sonnet-4-5	run-1777853456125	2026-05-04 00:10:56
Claude Opus 4.1	dev-anthropic-claude-opus-4-1	run-1777856235806	2026-05-04 00:57:15
GPT-5.4-reasoning	dev-gpt-54-reasoning	run-1777882982599	2026-05-04 08:23:02
GPT-5.5-reasoning	dev-gpt-55-reasoning	run-1777885083982	2026-05-04 08:58:03
GPT-5.4	dev-gpt-54-chat	run-1777882125342	2026-05-04 08:08:45
GPT-5.2	dev-gpt-52-chat	run-1777881635032	2026-05-04 08:00:35
o3	dev-gpt-5-reasoning	run-1777861576649	2026-05-04 02:26:16
GPT-5	dev-gpt-5-chat-jj	run-1777846519055	2026-05-03 22:15:19

Table 16: Identifiers preserved in the recorded result files. Configuration labels reproduce the experiment metadata. The endpoint-to-public-snapshot mapping is not independently verified.

Model	Pass	Rate	Stat	LLM	Scr	Lat
Claude Sonnet 4.5	69/206	33.5%	.309	.419	.385	27s
Claude Opus 4.1	61/206	29.6%	.306	.419	.383	41s
GPT-5.4-reasoning	58/206	28.2%	.230	.372	.336	22s
GPT-5.5-reasoning	57/206	27.7%	.300	.407	.374	32s
GPT-5.4	53/206	25.7%	.245	.333	.321	18s
GPT-5.2	47/206	22.8%	.197	.286	.269	16s
o3	46/206	22.3%	.214	.318	.290	67s
GPT-5	43/206	20.9%	.204	.277	.266	14s

Table 17: Recorded results across 206 scenarios. Stat averages the 163 scenarios with static checks, and LLM averages all 206. Scr is the mean scenario score, with equal weights when both components exist and the available component otherwise. Passing requires score ≥ 0.5 and all critical assertions to pass.

pass rates range from 20.9% to 28.2%. GPT-5.5-reasoning records 8.7 tool calls per scenario on average, compared with 3.6 for GPT-5.4-reasoning. These uncontrolled comparisons do not isolate the effects of model generation, reasoning mode, or tool-call count.

Static and LLM components. All configurations show differences between static and LLM component means (Table 17). Static means are computed only where the component exists, rather than including the runner’s default value for scenarios without static checks. Sonnet records a static mean of 0.309 and an LLM mean of 0.419, while GPT-5 records 0.204 and 0.277. These component means do not identify the mechanism of task failure or validate the grader. Critical checks can cause failure even when an aggregate score reaches the threshold.

C.3 Performance Across Task Categories

Table 18 and Figure 6 report variation across declared categories.

For Sonnet, the highest-performing categories include structured state changes: filters (80%), folders (88.9%), and settings (75%). Its calendar-write and orchestration pass rates are 16.7% and 17.6%,

Category	Son.	Opus	5.4-R	o3	GPT-5
filters	80	90	40	50	20
folders	89	56	89	44	44
settings	75	75	75	50	62
messages-write	73	67	47	47	53
contacts	17	17	17	17	17
messages-read	36	33	38	23	23
calendar-read	8	8	25	17	17
inbox-cleanup	44	22	22	22	33
todo	22	22	11	11	11
calendar-triage	38	25	38	25	25
meeting-prep	22	22	11	0	11
multi-domain	17	17	0	0	0
calendar-write	17	8	8	17	8
cross-domain	7	7	13	13	7
orchestration	18	18	12	12	12
boards	0	0	10	20	0

Table 18: Per-category pass rates (percent). Some write categories have high observed rates. Board tools are absent from the evaluated adapter.

Level	n	Son.	5.4-R	o3	GPT-5
D1	17	11.8	5.9	11.8	11.8
D2	121	42.1	37.2	30.6	28.1
D3	68	23.5	17.6	10.3	10.3

Table 19: Pass rates (percent) by assigned difficulty. Difficulty labels do not specify an exact number of calls or domains. Unavailable tools can affect low-difficulty tasks.

respectively. Full per-category results for all eight configurations appear in Appendix K. Performance differs across assigned difficulty groups: D1 at 6–12%, D2 at 28–42%, and D3 at 10–24%. These rates are not monotonic in the assigned level.

Recorded failure evidence. The available annotations do not support frequency estimates for a qualitative failure taxonomy. Appendix Q therefore reports only failure mechanisms grounded in recorded traces.

Difficulty Stratification. Table 19 breaks down performance by difficulty level.

C.4 Where Models Struggle

The recorded results vary across task categories but do not support attributing failures to a single model

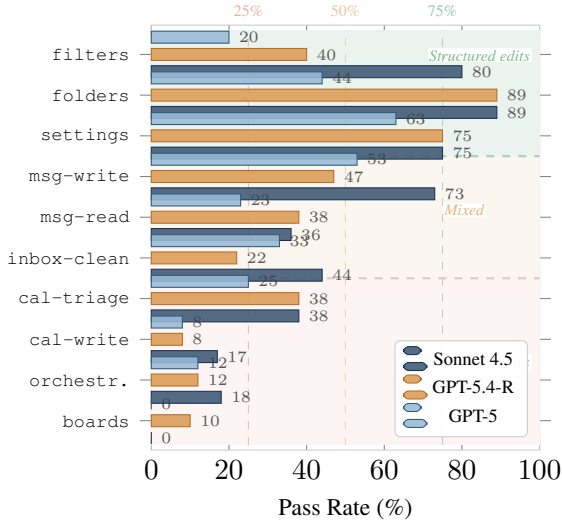


Figure 6: Per-category pass rates for three selected models. Shading groups structured edits, mixed categories, and other workflows. These descriptive groups do not define a read/write partition. Board results reflect an adapter without board tools.

capability.

Nested Object Construction. Calendar writes and recipient payloads have structured input requirements. Payload validity is distinct from satisfying a scenario’s requested time, recipients, or other constraints. We report no nesting-depth error percentages without a documented annotation procedure.

Result Piping. Workflows require using returned identifiers in later calls. The recorded OB-N520 trace correctly creates a parent folder, uses its ID for subfolders, and then uses subfolder IDs in batch moves.

Temporal Reasoning. Time-sensitive requests depend on the fixed corpus reference time and the mock API’s filtering semantics. No temporal-versus-nontemporal failure ratio is established by the available annotations.

Tool Call Efficiency. The eight recorded runs contain 378–1,782 tool calls per configuration. More calls do not guarantee a higher pass rate. Call count also reflects task mix, pagination, repeated retrieval, and zero-tool responses. Repetition alone is not evidence that a model is retrying an API error.

Unavailable Operations. There are 10 board-category scenarios and 19 scenarios declaring the boards domain. The adapter exposes no board

Domains	Son.	Opus	5.4-R	o3
1 ($n = 122$)	37.7	33.6	34.4	27.9
2 ($n = 53$)	34.0	28.3	26.4	20.8
3 ($n = 24$)	16.7	16.7	8.3	4.2
4 ($n = 7$)	14.3	14.3	0.0	0.0

Table 20: Pass rates (percent) by declared domain count. Cohort sizes sum to 206. Differences combine task composition, access restrictions, and model performance.

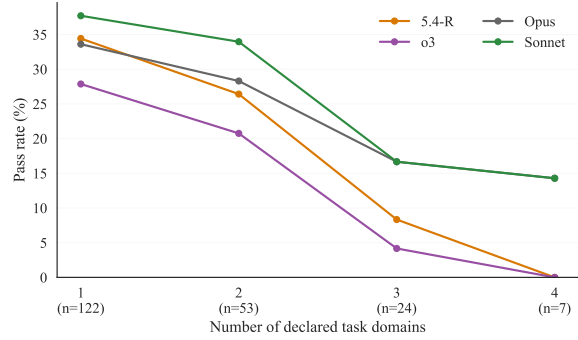


Figure 7: Pass rate by declared task-domain count, joined from scenario IDs. Cohort sizes are 122, 53, 24, and 7. Differences are descriptive associations, not an experimentally isolated cost per added domain.

tools. Failure on those tasks conflates tool availability with reasoning and cannot establish a board-specific capability deficit.

Domain Composition Scaling. Table 20 and Figure 7 quantify degradation by domain count.

Declared domain count is associated with different task mixes and small cohorts at the high end. These comparisons do not establish a causal decline of a fixed number of percentage points per added domain.

Inter-Model Agreement. To understand whether models fail on the same scenarios or exhibit complementary strengths, we compute pairwise agreement between four explicitly named models (Table 21).

For the named Sonnet/Opus/GPT-5.4-R/o3 cohort, 24/206 scenarios pass for every model, 117/206 pass for none, and 65/206 have differing outcomes. These sets are computed by scenario ID. Appendix S describes the cohort failed by all four.

Reasoning-Labeled Configurations. GPT-5.4-reasoning (28.2%) has a higher observed pass rate than GPT-5.4 (25.7%), a difference of 2.4 percentage points before rounding. Similarly, o3 passes 22.3% versus GPT-5’s 20.9%, with higher mean latency (67.17 vs. 13.59 seconds). GPT-5.5-

	Son.	Opus	5.4-R	o3
Sonnet 4.5	N/A	0.711	0.530	0.420
Opus 4.1	0.711	N/A	0.506	0.446
GPT-5.4-R	0.530	0.506	N/A	0.507
o3	0.420	0.446	0.507	N/A

Table 21: Jaccard similarity of recorded pass sets for the four named models. This selected cohort includes o3 and is not the top four by observed pass rate.

Model	1	2	3	5	7	10	15
Sonnet 4.5	0.0	11.7	22.8	30.1	32.0	32.0	33.5
Opus 4.1	0.0	6.8	15.0	21.8	25.2	26.7	29.6
GPT-5.4-R	0.0	7.8	15.0	22.8	25.2	27.7	28.2
GPT-5.5-R	0.5	8.7	13.6	18.4	21.4	26.2	27.7
GPT-5.4	0.0	14.6	20.4	23.3	25.2	25.2	25.7
GPT-5.2	0.5	11.2	18.0	20.4	21.8	22.3	22.8
o3	0.5	4.4	9.7	14.6	15.0	19.4	22.3
GPT-5	0.0	8.7	15.5	19.4	19.9	20.4	20.9

Table 22: Retrospective completed-pass rates (percent of all 206 scenarios) at iteration K . No lower-budget run is simulated. Recorded final outcomes are unchanged.

reasoning uses the most tools (8.7 on average) and passes 27.7%. These model comparisons do not isolate planning, schema recall, tool availability, or grading effects. Targeted interventions and a revised tool-access protocol would be needed to separate these explanations.

Differences Across GPT-5 Labels. The observed pass rates increase across GPT-5 (20.9%), GPT-5.2 (22.8%), and GPT-5.4 (25.7%), a total difference of 4.9 percentage points before rounding. All evaluated GPT-5 variants have lower observed pass rates than the two evaluated Anthropic models. These descriptive comparisons do not establish architectural causes or statistically significant differences between models.

C.5 Iteration Dynamics

We describe recorded iteration patterns across all eight configurations, including when passing trajectories terminate and how activity changes over a run. These retrospective summaries do not estimate performance under a different iteration budget.

C.5.1 Retrospective Completion by Iteration

Table 22 counts recorded trajectories that both passed at the original budget and completed within K iterations. This retrospective analysis is not a re-run under a lower iteration cap and does not bound such a rerun.

Figure 2 in the main paper visualizes these completion counts.

Model	1-pass	Later pass	Fail/calls	Fail/no calls
Sonnet 4.5	0	69	130	7
Opus 4.1	0	61	141	4
GPT-5.4-R	0	58	138	10
GPT-5.5-R	1	56	144	5
GPT-5.4	0	53	140	13
GPT-5.2	1	46	88	71
o3	1	45	136	24
GPT-5	0	43	92	71

Table 23: Trajectory counts partition all 206 outcomes. Iterations use observed activity when a stored count is missing. Failures are split by whether any tool call was recorded. A zero-tool response is not automatically a refusal.

By iteration five, the share of final passes already completed ranges from 30/46 for o3 (65.2%) to 40/43 for GPT-5 (93.0%). Sonnet completes 62/69 passes by that point (89.9%). Later completions contribute differently across models. These observations do not justify changing the iteration cap without a new experiment.

C.5.2 Per-Iteration Pipeline Breakdown

Figure 8 decomposes each iteration into pipeline stages, analogous to JQBench’s compile→execute→values progression (Verbruggen et al., 2026). It shows where the funnel narrows.

Activity decreases as trajectories terminate, so a smaller engaged fraction at later iterations is not itself an error. Active and engaged rates differ when a model is producing a final answer without a tool call. API failures are classified from structured response semantics, not words such as *failed* appearing in retrieved messages. Four exception records have zero stored iterations despite recorded calls. Their observed activity is reconstructed from call indices, with any unrecorded terminal step left unknown.

C.5.3 Self-Correction and Recovery

We decompose agent trajectories into four categories based on iteration count and outcome (Table 23).

Repeated tool calls. Repeated calls may represent pagination, retrieval of multiple entities, repeated mutations, or retries. Table 24 counts failed trajectories with calls that contain adjacent calls to the same tool with different serialized arguments. This observable pattern alone does not establish that an error occurred or that a model recognized it.

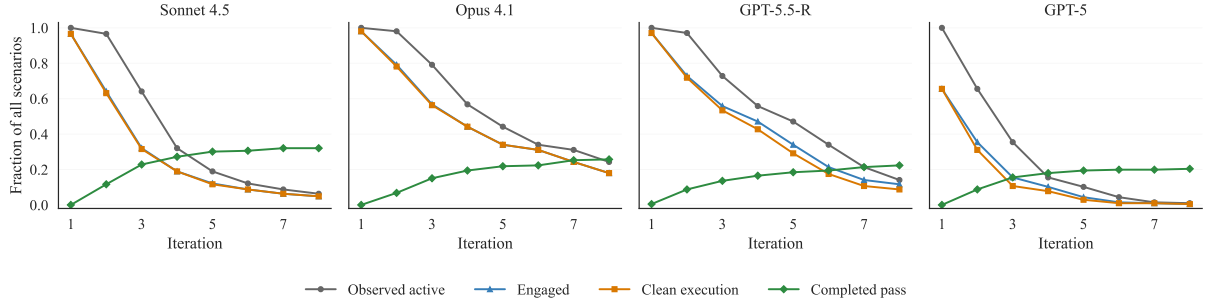


Figure 8: Per-iteration activity in four recorded runs. Active denotes observed ongoing activity. Engaged denotes at least one call in that iteration. Clean execution denotes engaged trajectories with no observed API failure in that iteration. Cumulative pass counts completed passing runs. All percentages use 206 scenarios. For aborted runs with missing terminal counts, Active is an observed lower bound reconstructed from recorded calls.

Model	Fail/calls	Repeated	%
Sonnet 4.5	130	72	55.4
Opus 4.1	141	93	66.0
GPT-5.4-R	138	71	51.4
GPT-5.5-R	144	101	70.1
GPT-5.4	140	40	28.6
GPT-5.2	88	21	23.9
o3	136	105	77.2
GPT-5	92	26	28.3

Table 24: Failed trajectories with adjacent same-tool calls using different arguments. The denominator is failed trajectories with recorded calls. This is a repetition measure, not an error-recovery rate.

Model	Calls/pass	Calls/fail	Iter/pass	Iter/fail
Sonnet 4.5	5.28	5.88	3.61	3.69
Opus 4.1	4.59	5.70	4.75	5.85
GPT-5.4-R	3.00	3.84	4.00	4.80
GPT-5.5-R	7.60	9.05	4.81	4.56
GPT-5.4	2.70	2.64	2.98	3.12
GPT-5.2	3.34	1.42	3.13	2.10
o3	4.63	7.06	5.63	7.79
GPT-5	3.02	1.52	3.21	2.12

Table 25: Mean recorded tool calls and observed iterations, grouped by outcome. The groups differ in task composition. These differences are descriptive, not evidence of a causal effect of extra calls.

Zero-tool responses. Table 23 separates failed responses with no recorded calls. Their content may be a refusal, an unsupported answer, or a report of missing capabilities. Those interpretations require inspecting the response rather than inferring intent from call count.

C.5.4 Pass vs. Fail Behavioral Signatures

Table 25 reveals distinct behavioral patterns between passing and failing trajectories.

Passing and failing cohorts contain different scenarios. Additional calls can reflect useful work, repetition, or harder tasks, while zero-tool failures lower the failure-group mean. The table does not

Model	1	2	3-4	5-7	8+
Sonnet 4.5	0/7	24/67	32/93	10/26	3/13
Opus 4.1	0/4	14/39	26/72	12/41	9/50
GPT-5.4-R	0/11	16/47	27/74	9/41	6/33
GPT-5.5-R	1/6	17/50	16/53	10/68	13/29
GPT-5.4	0/13	30/91	17/71	5/25	1/6
GPT-5.2	1/72	22/69	17/50	5/10	2/5
o3	1/25	8/18	16/37	6/38	15/88
GPT-5	0/71	18/62	19/52	4/19	2/2

Table 26: Recorded passes and scenario counts by observed iteration bucket. Small high-iteration cohorts can have unstable rates. These are not rerun results at different budgets.

identify an optimal tool-use policy.

C.5.5 Iteration Efficiency by Bucket

We group scenarios by the number of iterations the agent used and compute per-bucket pass rates (Table 26).

Bucket membership is determined after execution. It cannot establish an optimal iteration budget, because models and scenarios enter the buckets for different reasons.

C.5.6 Score Distribution

Figure 9 shows recorded overall scores. A score of at least 0.5 is necessary but not sufficient to pass because critical assertions are also required.

Scores alone do not distinguish API failure, incomplete work, or critical-check failure. The recorded OB-H307 example reaches 0.50 yet fails its critical LLM rubric.

C.6 Graduated Execution Pipeline

Table 27 reports separate scenario-level indicators. These are not all nested stages: a trajectory can produce an answer after an API error, and a stored pass can occur without a recorded call. Clean execution is an API-behavior indicator, not evidence

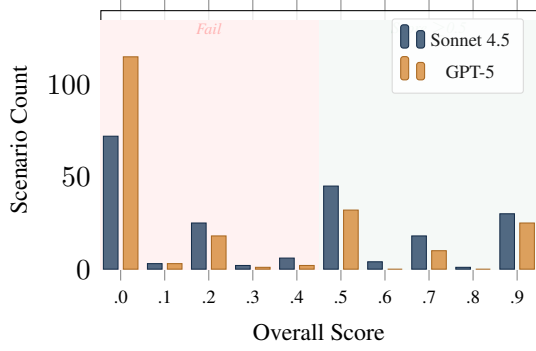


Figure 9: Score distribution for Sonnet and GPT-5 across 206 scenarios. Bins have width 0.1, and the last includes both endpoints [0.9, 1.0]. Shading separates scores below 0.5 from those at or above it. The latter can still fail critical assertions. Both distributions are bimodal, with many scenarios near 0 or above 0.5 and few intermediate cases.

Model	Engage	Clean	Answer	Pass
Sonnet 4.5	96.6	94.7	100.0	33.5
Opus 4.1	98.1	96.6	100.0	29.6
GPT-5.4-R	95.1	88.8	97.1	28.2
GPT-5.5-R	97.1	88.3	99.0	27.7
GPT-5.4	93.7	89.3	100.0	25.7
o3	87.9	79.1	78.6	22.3
GPT-5.2	65.0	58.7	100.0	22.8
GPT-5	65.5	57.8	100.0	20.9

Table 27: Scenario indicators from recorded runs (percent of 206). Engagement requires a recorded tool call. Clean execution additionally requires no observed API failure. Answer excludes runner error and max-iteration sentinel strings. Pass uses the recorded grade.

of task correctness.

Execution classification uses top-level error envelopes, nonempty batch-failure lists, and failed mutation return values. This avoids treating text such as `failed: []`, nullable calendar fields, or message bodies containing error-related words as failures. The classification affects execution statistics but does not recompute task grades.

C.7 Tool Call Success and Error Feedback

C.7.1 Execution Classification

Nonempty top-level error envelopes and batch failure lists count as observed API failures. A mutation returning `false` or `null` also counts as a failure. An absent-object lookup is recorded separately from mutation failure. These rules measure the API result, not whether arguments matched the user’s intended task.

Tool results were truncated at 8,000 characters. A truncated array is not classified as a failure merely because it cannot be parsed as complete

Model	Calls	Success %	First %	Failures
Sonnet 4.5	1170	99.7	100.0	4
Opus 4.1	1107	99.7	100.0	3
GPT-5.4-R	743	96.8	100.0	24
GPT-5.5-R	1782	94.1	100.0	106
GPT-5.4	547	95.4	100.0	25
o3	1342	95.8	100.0	57
GPT-5.2	383	92.4	100.0	29
GPT-5	378	92.1	100.0	30

Table 28: Tool-call execution statistics using structured failure evidence. Calls are individual invocations. First-call rates use trajectories with at least one call. Stored grading outcomes are not recomputed.

Model	Failures	Next calls	Success %	Same %
Sonnet 4.5	4	1	100.0	0.0
Opus 4.1	3	0	N/A	N/A
GPT-5.4-R	24	17	52.9	47.1
GPT-5.5-R	106	98	14.3	86.7
GPT-5.4	25	17	5.9	94.1
o3	57	46	30.4	71.7
GPT-5.2	29	18	33.3	66.7
GPT-5	30	14	0.0	100.0

Table 29: Next-call behavior after observed API failures. Denominators exclude terminal failures with no following call. Same-tool continuation and successful next execution do not by themselves establish recovery of the original task.

JSON. The analysis records truncation and distinguishes unclassifiable results from observed success or failure. Successful calls can still return incomplete data, use the wrong filter, or perform an unintended operation.

For example, Sonnet has four observed failures among 1,170 calls, a 99.66% rate of calls without observed API failure. Its much lower scenario pass rate demonstrates the distinction between invoking an API and completing a task. The former claim that every Sonnet batch move failed arose from treating an empty `failed` array as an error.

C.7.2 Error Feedback Utilization

For an observed failure followed by another call in the same trajectory, Table 29 records whether that next call executes successfully and whether it uses the same tool. Calls issued together in one model iteration cannot establish response to feedback. These descriptive next-call rates must not be interpreted as a causal measure of learning from an error.

With few observed failures for some models, recovery fractions have small denominators. The data do not establish that late execution errors or repeated attempts explain most task failures.

C.8 Complexity Correlations

Figure 10 joins every recorded result to its scenario definition by ID before forming feature cohorts. Each signed difference is the pass rate for scenarios with the feature minus the rate without it. Signs are preserved. These are descriptive associations among overlapping cohorts, not controlled causal effects.

Declared domain count, assigned difficulty, assertion count, and observed tool-call count describe different aspects of a trajectory. In particular, observed call count is not a known minimum number of calls required. Task categories and available tools can confound these comparisons.

Figure 11 presents the declared-domain cohorts directly. The four-domain group contains only seven scenarios. No fixed per-domain penalty or causal claim about assertion count is inferred.

D Supplementary Analysis

D.1 Structural Requirements in Email Tasks

The benchmark represents three recurring structural requirements:

Nested entity structures. Unlike flat key-value stores, email entities have deeply nested types. A Message has `from.emailAddress.address` (3 levels), `flag.flagStatus`, and `Recipient[]` arrays. Calendar events add `start.dateTime/start.timeZone` and `EventAttendee[]` with nested `status.response` fields. An incorrect field path can make a call invalid or direct it to the wrong value.

Temporal reasoning. Many scenarios require date arithmetic relative to the reference time and understanding temporal relationships (“tomorrow”, “last 3 days”, “within 30 minutes”).

Multi-step dependencies. Write workflows have ordering constraints, such as creating a folder before moving emails or retrieving an attendee list before forwarding. Failure at one step can propagate through the remaining workflow.

D.2 Implications for Agent Design

The recorded failures motivate several interventions for future evaluation:

Schema grounding. Explicit schema documentation and structured output constraints could help

Model	Calls	Iterations	Latency (s)
Sonnet 4.5	5.68	3.66	27.06
Opus 4.1	5.37	5.52	40.56
GPT-5.4-R	3.61	4.58	21.84
GPT-5.5-R	8.65	4.63	31.69
GPT-5.4	2.66	3.09	17.80
GPT-5.2	1.86	2.33	15.84
o3	6.51	7.31	67.17
GPT-5	1.83	2.35	13.59

Table 30: Per-scenario means from recorded runs. Observed iterations use recorded call indices where a stored iteration count is missing. Exact unrecorded terminal activity remains unknown.

with payload construction, but their effect needs separate evaluation.

Pipeline planning. Explicit dependency planning is a candidate intervention for incomplete workflows. Its effect is not measured here. A “plan-then-execute” architecture that first generates a dependency DAG (e.g., “folder ID from step 1 feeds step 3”) could address this.

Typed validation. Runtime type checking or schema validation before API dispatch could identify malformed payloads. Whether such a layer improves task completion remains an empirical question.

Domain-specialized prompting. Prompting interventions require evaluation with an explicit tool manifest. Board tools must first be made available before testing whether prompting improves board-task performance.

D.3 Recorded Resource Use

The result artifacts do not include token-usage data. Table 30 therefore reports recorded calls, observed iterations, and wall-clock latency.

D.4 Efficiency–Accuracy Tradeoff

Figure 3 in the main paper plots the empirical latency–accuracy frontier. No other configuration is both at least as accurate and at least as fast as a frontier point, with one strict improvement.

Efficiency is defined here as pass-rate percentage points divided by mean scenario latency in seconds. Using unrounded recorded latencies, GPT-5 has a ratio of 1.5355 pp/s, GPT-5.4-R has 1.2890 pp/s, and Sonnet has 1.2377 pp/s. This ratio is one preference rule, not a universal ranking.

Sonnet has the highest observed accuracy and is on the Pareto frontier. There is no unique best frontier model without specifying an accuracy/latency

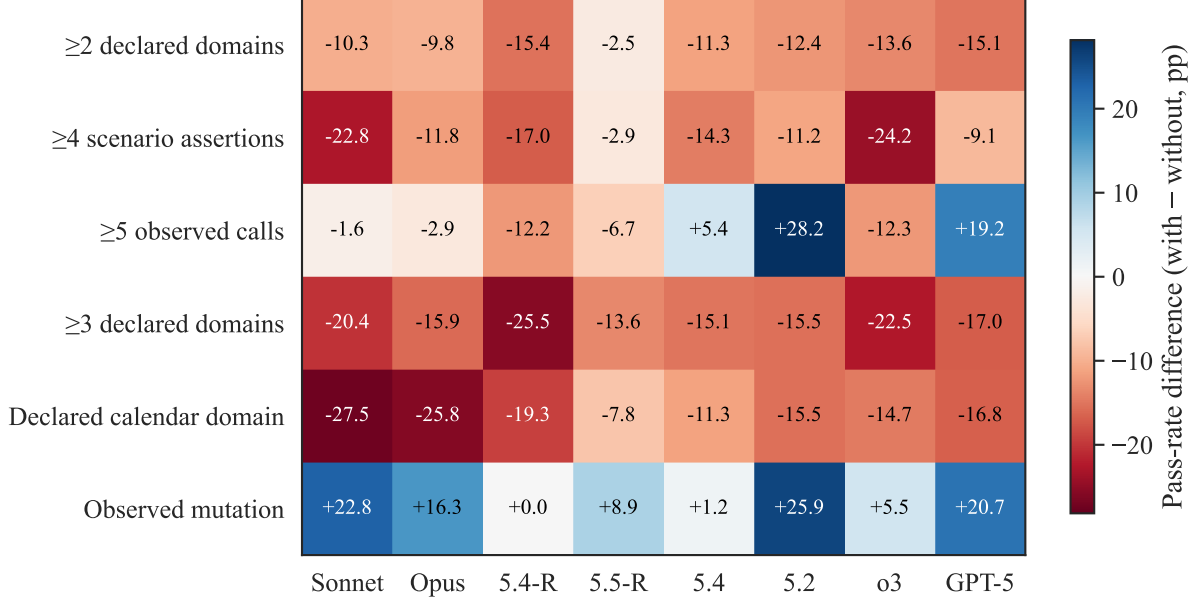


Figure 10: Signed pass-rate differences (percentage points) for documented complexity proxies. Scenario metadata are joined by ID, and execution-derived features describe observed behavior. Cohort counts and feature definitions accompany the regenerated analysis.

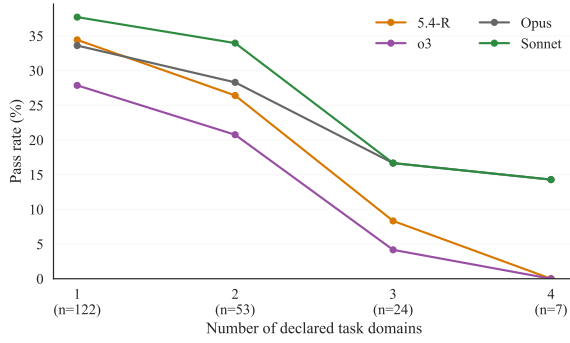


Figure 11: Pass rates grouped by the number of declared task domains: 122, 53, 24, and 7 scenarios. The grouping does not imply a prescribed order of API calls.

tradeoff. o3’s mean latency is 67.1671 s versus Sonnet’s 27.0616 s, a $2.48\times$ ratio.

D.5 Corpus Realism vs. Scale

Our seed corpus (65 messages) is deliberately small compared to production mailboxes. **Advantages:** fixed mock data, local API operations, and high interconnection density. Model inference and LLM grading are not deterministic. **Limitations:** limited pagination pressure, reduced needle-in-haystack difficulty, and no rate limiting. Future work could introduce scaled variants with 1,000+ messages.

D.6 Variance and Reproducibility

The analysis contains one complete result file per reported configuration. The available artifacts do not provide multiple runs per configuration, so we do not report replicate variance or attribute observed variance to the judge.

Static checking is deterministic for fixed inputs, but model outputs and LLM judgments may vary. Figure 5’s Wilson intervals quantify the separate, explicitly stated scenario-sampling uncertainty.

E Protocol-Specific Constraints

Simulated environment. The mock API cannot capture network latency, rate limiting, eventual consistency, or OAuth authentication flows present in production deployments.

Corpus scale. The seed corpus (~65 messages) is smaller than production mailboxes (which may have thousands of messages), limiting evaluation of pagination handling, search precision at scale, and information retrieval over large collections.

Single-turn evaluation. Real users iteratively refine agent outputs through multi-turn correction (“no, I meant next Tuesday”). Our single-turn protocol does not capture this interactive loop.

English-only. All corpus content and queries are in English. Multilingual email management (common in global enterprises) is not evaluated.

Single-user baseline. All reported scenarios run

against Kaminski’s corpus, including tasks naming another user. Cross-persona generalization is not evaluated. Missing board, directory, and contact-group tools limit capability attribution for failures on those operations.

Grader limitations. LLM judgments may vary and miss domain-specific nuances. Available artifacts do not establish repeated-run stability or expert agreement. Equal weighting and critical checks do not remove this uncertainty. Answer-pattern and snapshot-check limitations also affect validity.

No multi-agent evaluation. We evaluate single-agent performance only. Multi-agent delegation patterns (e.g., a planning agent dispatching to specialist sub-agents) may achieve different results.

F Data Provenance Details

EmailBench uses fabricated content inspired by publicly available Enron email data (Klimt and Yang, 2004). It includes no original email bodies or private mailbox exports. Message bodies, calendar events, tasks, and other scenario content are synthetic. Publicly documented names, email addresses, roles, and organizational relationships remain, so the corpus should not be described as containing no real personal information. The Enron corpus was released by FERC under public disclosure mandate and has been widely used in email research (Klimt and Yang, 2004; Zhang et al., 2021). Public availability alone does not establish permission to reuse or redistribute every source item or identifier.

The available benchmark artifacts expose only aggregate intent-frequency counts from the prototype telemetry referenced in §A.3. They do not establish how the source telemetry was collected, de-identified, or retained, so we make no stronger privacy claim about the unavailable source records.

G Reproducibility and Availability

The benchmark environment is self-contained, while model and judge inference requires external endpoints. The available artifact supports the following forms of inspection and replay:

- **Self-contained state.** The mock API runs in memory over a frozen corpus and does not require a live email provider.
- **Deterministic corpus.** The seed corpus is a

typed constant, and corpus initialization is deterministic.

- **Grading scripts.** The 258 static assertions are encoded as TypeScript predicates or answer checks. The 211 LLM rubrics and the judge prompt are versioned alongside the benchmark.
- **Agent adapter.** The adapter exposes the evaluated tool schemas and permits replacement of the model endpoint without changing benchmark state.
- **Recorded evidence.** Result files preserve final answers, abbreviated tool traces, grades, internal endpoint IDs, run IDs, and timestamps. They do not preserve an exact source-commit and tool-schema manifest. Offline validation reproduces the stored component scores, combined scores, and critical-check pass flags for all 1,648 saved scenario results.

The reviewed repository contains the benchmark code, corpus, scenarios, grading scripts, and baseline agent implementations. An explicit license covering every artifact was not located, so we make no blanket licensing claim. Model and judge execution also requires access to the configured hosted services and credentials.

H Future Evaluation

Future evaluations should separate environment access from model capability by exposing every operation required by the scenarios and selecting the declared user corpus. Repeated runs and expert validation of the LLM judge are also needed. Additional benchmark variants could study larger corpora, multi-turn interaction, provider-specific implementations, schema-grounding interventions, and multi-agent execution. These extensions require new experiments and are not implied by the reported baseline.

I Batch 4 Gap-Filling Details

After the first three batches (122 scenarios), we compute per-category coverage and generate 84 additional scenarios. Table 31 shows the targeted additions.

Type	Count	%
return-contains	123	47.7
state-check	101	39.1
exists	32	12.4
sent-message	2	0.8
count	0	0.0
not-exists	0	0.0
property	0	0.0
moved-message	0	0.0
deleted	0	0.0
Total	258	100.0

Table 32: Code-derived distribution of 258 static assertions. Five supported types have no instances, and four types are used.

Category	Before	Added	Final
meeting-prep	1	8	9
calendar-triage	2	6	8
settings	2	6	8
folders	3	6	9
inbox-cleanup	3	6	9
calendar-write	7	5	12
contacts	7	5	12
todo	4	5	9
boards	5	5	10
filters	5	5	10
+ 6 other categories	N/A	27	N/A

Table 31: Gap-filling in Batch 4. Categories with fewer than eight scenarios received targeted additions to establish minimum category coverage.

J Static Assertion Distribution

Across 206 scenarios, there are 258 static assertions (mean 1.252). Forty-three scenarios have none. Counts are regenerated from scenario definitions and checked against the recorded assertion outcomes.

K Full Per-Category Results

Table 33 presents pass rates across all 16 categories for all eight configurations.

Category	Son.	Opus	5.4-R	5.5-R	5.4	5.2	o3	GPT-5
filters	80.0	90.0	40.0	40.0	30.0	50.0	50.0	20.0
folders	88.9	55.6	88.9	66.7	55.6	44.4	44.4	44.4
settings	75.0	75.0	75.0	50.0	62.5	62.5	50.0	62.5
messages-write	73.3	66.7	46.7	60.0	46.7	53.3	46.7	53.3
contacts	16.7	16.7	16.7	16.7	25.0	8.3	16.7	16.7
messages-read	35.9	33.3	38.5	25.6	35.9	28.2	23.1	23.1
calendar-read	8.3	8.3	25.0	25.0	16.7	8.3	16.7	16.7
inbox-cleanup	44.4	22.2	22.2	55.6	11.1	22.2	22.2	33.3
todo	22.2	22.2	11.1	0.0	0.0	0.0	11.1	11.1
calendar-triage	37.5	25.0	37.5	37.5	37.5	25.0	25.0	25.0
meeting-prep	22.2	22.2	11.1	22.2	22.2	11.1	0.0	11.1
multi-domain	16.7	16.7	0.0	0.0	0.0	8.3	0.0	0.0
calendar-write	16.7	8.3	8.3	25.0	16.7	16.7	16.7	8.3
cross-domain	6.7	6.7	13.3	26.7	20.0	13.3	13.3	6.7
orchestration	17.6	17.6	11.8	11.8	17.6	11.8	11.8	11.8
boards	0.0	0.0	10.0	0.0	0.0	0.0	20.0	0.0

Table 33: Pass rates by declared scenario category for all eight configurations. Rows are regenerated by joining recorded outcomes to scenario IDs.

L Complete Scenario Examples

These queries and rubrics come from the evaluated scenario definitions. Static checks below describe executable conditions rather than relying only on their labels.

L.1 OB-H312: Sender Percentages

Category: messages-read. **Difficulty:** D2. **Domains:** messages.

Query: *For each person who emailed me, calculate what percentage of their emails to me are unread. Show only people with >50% unread rate, sorted by unread percentage descending.*

Static assertions. Both static assertions are critical. The answer must contain the literal string `\d+(\.\d+)?%` and the case-insensitive substring `lay`. Although the first pattern has regex syntax, the executed checker uses substring matching. This is a grading limitation, not a desired output format. The recorded Sonnet run makes two message-list calls in three iterations and scores 0. The percentage assertion fails despite an answer containing 100.0%. Its LLM judgment separately finds the sender analysis incorrect.

LLM rubric (critical, correctness). Must compute unread/total ratio per sender from inbox messages. Senders with >50% unread: Lay (3 emails, most unread since appearing at hoursAgo 9,46,70), Belden (2 emails both likely unread), Mark (2 emails). Must show exact percentages and sort descending.

L.2 OB-N520: Folder Hierarchy

Category: folders. **Difficulty:** D3. **Domains:** folders, messages.

Query: *Create a folder hierarchy: 'Projects' with subfolders 'Raptor', 'Dynergy', and 'FERC'. Then move all emails mentioning each project keyword to the appropriate subfolder.*

Static assertions. All five static assertions are critical: folders named Projects, Raptor, Dynergy, and FERC must exist, and at least three move records must exist. These predicates check names and move count, not parent-child relationships or keyword routing.

LLM rubric (critical, action-taken). Must create parent 'Projects' folder, then 3 subfolders under it. Then scan all emails for 'Raptor', 'Dynergy', 'FERC' keywords in subject/body and move matches to the right subfolder. Report move counts per folder.

L.3 OB-H307: Ordered Inbox Triage

Category: inbox-cleanup. **Difficulty:** D3. **Domains:** messages, folders.

Query: *Set up an email triage system: (1) Create folders 'Action Required', 'FYI Only', and 'Automated'. (2) Move all emails from automated senders (enrononline, legal-tickets, gasbank) to 'Automated'. (3) Move all high-importance emails to 'Action Required'. (4) Move all remaining unread emails to 'FYI Only'. Do NOT move any emails that are already in Sent Items.*

Static assertions. All six static assertions are critical. Folders named Action Required, FYI Only, and Automated must exist. At least four move records must involve the selected automated IDs (msg-003/016/024/039/007/012), and at least four must involve selected high-importance IDs (msg-001/006/008/013/027/036). None may involve Sent Items IDs msg-041 through msg-055. The two move-count checks do not test destinations. Ordering, remaining unread messages, and reporting are judged by the rubric.

LLM rubric (critical, action-taken). Steps must execute in order: automated first (so msg-024/msg-039 go to Automated despite being high-importance), then high-importance, then remaining unread. Sent items must be excluded. Must report counts per folder.

M Agent System Prompt

All eight reported configurations receive the identical system prompt below, with no few-shot examples, chain-of-thought instructions, or tool documentation beyond the JSON schema definitions:

```
You are an AI assistant that manages Microsoft Outlook. You have access to tools for managing emails, calendar, contacts, folders, rules, todo lists, and settings.

When the user asks you to do something:
1. Think about which tools you need to call
2. Call the appropriate tools to gather data or perform actions
3. Provide a clear, complete answer based on the tool results

Guidelines:
- For read queries: call the relevant list/search/get tools, then summarize the results clearly
- For write actions: call the appropriate create/update/delete/send tools
- For multi-step tasks: break them down and execute each step
- Always provide specific data (counts, names, dates) from tool results -- never make up information
- When listing items, include key details (subjects, senders, dates, etc.)
```

The prompt is deliberately minimal (148 tokens) to characterize behavior without task-specific prompt engineering. The agent receives no information about the corpus, entity IDs, API schemas beyond tool-definition JSON, or expected output format.

Agent Loop Configuration. Maximum iterations: 15. The checked-in agent request omits temperature and sets `max_completion_tokens=4096`. Its 180-second bridge subprocess limit applies per request, not per scenario. Tool results are truncated to 8,000

characters. When no tool calls are returned by the model, the loop terminates and the text response is taken as the final answer. Judge requests use `max_completion_tokens=2048` and a 120-second subprocess limit. The underlying bridge uses a 120-second HTTP timeout.

N Complete Evaluated Tool Surface

The adapter exposes 46 function tools with JSON-schema parameters. Table 34 is regenerated from these definitions. The 119-method operational API contains additional functionality, but no board, directory, or contact-group tool is supplied in the reported runs.

Interface scope. The function schema is the model’s direct contract. The full API-method count cannot substitute for exposed-tool count, and results on omitted operations require an access-limited interpretation.

O Seed Corpus Details

The corpus is anchored at reference time **2001-10-15T10:00:00Z** (Monday morning). All temporal offsets (`hoursAgo`, `daysAgo`, `daysFromNow`) are computed deterministically relative to this anchor.

O.1 Persona

The primary user is **Vince Kaminski** (`vkamins@enron.com`), VP & Head of Research at Enron Corp, located on Floor 19 of the Enron Tower. The corpus includes 16 users total: 10 Enron executives/employees, 2 Dynegy external partners, 3 system/bot accounts, and Kaminski himself.

O.2 Message Distribution

The corpus defines 40 recent incoming messages, 15 sent messages, and 10 older messages. Actual `parentFolderId` values give 50 Inbox messages, 15 Sent Items messages, and none in Archive: older-message construction retains the Inbox default.

Folder metadata differ from those object counts (for example, Inbox reports 42 and Sent Items reports 120). The corpus and frozen-read changelog implementation are preserved here. These discrepancies limit count-based and mutation-followup interpretations.

Key conversation threads: `conv-raptor` (4 messages about SPE financing), `conv-weather` (2 messages on weather derivatives), `conv-ferc` (2 messages on FERC filings), `conv-montecarlo` (2 messages on simulation parameters), `conv-dynegy` (2 messages on joint venture).

O.3 Calendar Events

The 15 events span October 8–23, 2001, around the October 15 reference time:

- **Before the reference time:** Project Raptor Kick-off (Oct 8) and Q3 Trading Retrospective (Oct 12).
- **Oct 15:** Research Team Standup, 1:1 with Skilling, Risk Committee Meeting, and Gas Volatility Sync.
- **Oct 16:** FERC Compliance Review, Dynegy Partnership Review, and 1:1 with Kitchen.
- **Oct 17–20:** Focus Time and Project Raptor Deep Dive (Oct 17), Team Lunch at Pappadeaux (Oct 18), Trading Division All-Hands (Oct 19), and Dynegy Credit Review (Oct 20).
- **Oct 22–23:** Energy Risk Conference.

FERC Compliance Review and Dynegy Partnership Review overlap on October 16 from 11:00 to 12:00 UTC. The date helper calls `setUTCHours(hour, 0, 0, 0)`, which truncates fractional hours. Consequently, `evt-001`, `evt-002`, `evt-005`, and `evt-010` have zero duration. Some duration and free-slot rubrics expect fractional-hour boundaries instead, so affected timing-task grades have fixture or rubric validity limitations. The reported results preserve the evaluated fixture and saved grades.

O.4 Other Entities

- **15 contacts:** 10 Enron executives, 2 Dynegy partners, 3 institutional (Enron Treasury, NYMEX Operations, FERC Filings)
- **3 todo lists:** “Tasks” (4 tasks), “Research Items” (3 tasks), “Project Raptor Backlog” (3 tasks), 10 tasks total spanning VaR model review, forward curve updates, and SPE valuation
- **2 boards:** “Project Raptor” (4 tasks across To Do/In Progress/Done) and “Q4 Research Goals” (3 tasks)
- **6 mail folders:** Inbox, Sent Items, Drafts, Deleted Items, Archive, Junk Email
- **1 mail filter:** Auto-move EnronOnline trade confirmations
- **3 contact groups:** Research Team, Risk Committee, Regulatory Affairs
- **11 attachments:** Spreadsheets, PDFs, and documents across 9 messages

Tool name	Domain	R/W	Parameters
messages_list	messages	R	top, filter, orderBy, skip, folderId
messages_get	messages	R	id
messages_search	messages	R	query, top
messages_count	messages	R	filter
messages_send	messages	W	subject, body, toRecipients, ccRecipients, importance
messages_createDraft	messages	W	subject, body, toRecipients
messages_update	messages	W	id, patch
messages_move	messages	W	id, destinationFolderId
messages_delete	messages	W	id
messages_reply	messages	W	id, comment
messages_replyAll	messages	W	id, comment
messages_forward	messages	W	id, toRecipients, comment
messages_batchMove	messages	W	ids, destinationFolderId
folders_list	folders	R	N/A
folders_get	folders	R	id
folders_getByName	folders	R	name
folders_create	folders	W	displayName, parentFolderId
folders_rename	folders	W	id, newName
folders_delete	folders	W	id
filters_list	filters	R	N/A
filters_create	filters	W	name, isEnabled, criteria, actions
filters_update	filters	W	id, patch
filters_delete	filters	W	id
calendar_list	calendar	R	top, startDateTime, endDateTime
calendar_get	calendar	R	id
calendar_create	calendar	W	subject, start, end, location, attendees, body, isAllDay
calendar_update	calendar	W	id, patch
calendar_delete	calendar	W	id
calendar_accept	calendar	W	id, comment
calendar_decline	calendar	W	id, comment
contacts_list	contacts	R	top
contacts_search	contacts	R	query
contacts_create	contacts	W	displayName, givenName, surname, emailAddresses, companyName, department, jobTitle
contacts_delete	contacts	W	id
todo_lists	todo	R	N/A
todo_createList	todo	W	displayName
todo_tasks	todo	R	listId
todo_createTask	todo	W	listId, title, body, dueDateTime, importance
todo_updateTask	todo	W	listId, taskId, patch
todo_deleteTask	todo	W	listId, taskId
settings_getVacationResponder	settings	R	N/A
settings_setVacationResponder	settings	W	enabled, message, startDateTime, endDateTime, sendToContactsOnly
settings_	settings	R	N/A
getSenderClassifications	settings	W	senderEmailAddress, classifyAs
createSenderClassification	settings	R	N/A
settings_getLabels	settings	R	N/A
getCurrentUser	identity	R	N/A

Table 34: All 46 evaluated tools. R denotes read and W denotes write. Names and parameters are shown as exposed. These abbreviated payloads can differ from the full operational API.

P LLM-Judge Protocol

P.1 Judge System Prompt

The checked-in evaluator uses judge deployment `dev-gpt-5-chat-jj` and does not specify temperature. It receives the following system prompt:

```
You are an expert evaluator for an AI email assistant benchmark.
You will be given:
1. The user's original query
2. The agent's final answer
3. A list of tool calls the agent made
4. A rubric describing what to check

Score the agent's answer on a scale of 0.0 to 1.0:
- 1.0 = fully satisfies the rubric
- 0.5 = partially satisfies
- 0.0 = completely fails

Respond with ONLY a JSON object:
{"score": <number>, "reason": "<brief explanation>"}
```

P.2 Seven Semantic Dimensions

Each LLM assertion targets one of seven semantic aspects:

1. **Correctness:** Is the answer factually accurate given the corpus data? (e.g., correct counts, names, dates)
2. **Completeness:** Does the answer include *all* required information, or does it omit key details?
3. **Relevance:** Is the response pertinent to the query, without extraneous tangents?
4. **Format:** Is the output well-structured and clearly presented? (e.g., tables for tabular data, bullet points for lists)
5. **Safety:** Does the agent avoid harmful, unauthorized, or destructive actions beyond what was requested?

Name	Email	Dept.	Title
Jeff Skilling	jeff.skilling@enron.com	Executive	President & COO
Louise Kitchen	louise.kitchen@enron.com	Trading	Pres., EnronOnline
Sally Beck	sally.beck@enron.com	Operations	VP Operations
John Lavorato	john.lavorato@enron.com	Trading	CEO, Enron Americas
Richard Shapiro	richard.shapiro@enron.com	Gov. Affairs	VP Gov. Affairs
Steven Kean	steven.kean@enron.com	Executive	VP & Chief of Staff
Mark Taylor	mark.taylor@enron.com	Legal	VP & Gen. Counsel
Kenneth Lay	kenneth.lay@enron.com	Executive	Chairman & CEO
Stinson Gibner	stinson.gibner@enron.com	Research	Senior Analyst
Zimin Lu	zimin.lu@enron.com	Research	Quant. Analyst
Tim Belden	tim.belden@dynegy.com	Trading	VP Western Trading
Rebecca Mark	rebecca.mark@dynegy.com	Development	VP Bus. Dev.
EnronOnline Sys.	system@enrononline.com	N/A	(bot)
Legal Ticketing	legal-tickets@enron.com	N/A	(bot)
GasBank Platform	alerts@gasbank.enron.com	N/A	(bot)

Table 35: All 15 non-self users in the corpus, spanning executives, analysts, external partners, and system accounts.

6. **Action-taken:** Did the agent perform the correct mutations? (Used for write scenarios where the rubric checks that specific API calls were made correctly.)
7. **No-hallucination:** Does the answer avoid fabricating data not present in the corpus? (e.g., inventing email subjects, contact details, or meeting times)

Rubric Design. Each rubric is hand-written by the scenario author and specifies exact entities, IDs, and expected relationships. For example, a rubric for OB-H300 (cross-domain correlation) explicitly names the expected intersection of senders and attendees: “Key people: Skilling (6 emails, in Research Standup/1:1/Risk Committee), Beck (6 emails, in Risk Committee), Kitchen (5 emails, in many events)...Must NOT include EnronOnline/GasBank system accounts.”

P.3 Judge Input Format

For each LLM assertion, the judge receives a user message containing:

```
## Query
<scenario query>

## Agent's Answer
<final text answer from agent>

## Tool Calls Made
tool_name(args...) -> result...
tool_name(args...) -> result...
(arguments: 100 chars; result: 200 chars)

## Rubric to evaluate
**<label>**: <rubric text>

Respond with ONLY:
{"score": <0.0-1.0>, "reason": "..."}

```

The evaluated judge receives at most 100 characters of arguments and 200 of results per call. This incomplete trace can limit its ability to verify execution details.

Iter	Recorded calls and results (summary)
1	<code>folders_create</code> : Projects, returning folder-5001.
2	Three <code>folders_create</code> calls: Raptor, Dynegy, and FERC, all with parent folder-5001 and returning folder-5002/5003/5004.
3	Three <code>messages_search</code> calls, one per keyword.
4	Three <code>messages_batchMove</code> calls. Results contain 6, 8, and 8 succeeded IDs and empty failed lists.
5	Final answer reports the hierarchy and move counts.

Table 36: Recorded OB-N520 success: five iterations, ten calls, and 34.886 s. Static, LLM, and overall scores are all 1.0. The scenario passes.

Iter	Recorded calls and results (summary)
1	Creates Action Required, FYI Only, and Automated folders (folder-5001/5002/5003).
2–5	Retrieves Sent Items, lists messages, searches three automated senders, lists high-importance messages, and attempts an unread filter.
6	Moves three EnronOnline matches in a batch and two other automated messages individually.
7	Moves two further automated messages and six high-importance messages.
8–11	Lists messages, attempts a folder filter, retrieves four folders, and lists folders.
12	Creates another set of the same three named folders.
13–15	Continues inbox listing with <code>folderId</code> and <code>skip</code> arguments until the iteration limit.

Table 37: Recorded OB-H307 failure: 15 iterations, 29 calls, and 88.868 s. All six critical static checks pass with a static score of 1.0. The critical LLM rubric fails with a score of 0.0, yielding an overall score of 0.50 and a failing outcome.

Q Example Agent Trajectories

These summaries use recorded Claude Sonnet 4.5 tool calls from the result file with timestamp 2026-05-04T00:10:56.125Z. Iterations below are one-based. Arguments and results are abbreviated. The files preserve final answers and grading details but not every intermediate assistant message or untruncated response.

Q.1 Successful Trace: OB-N520 (folders, D3)

The query creates Projects with Raptor, Dynegy, and FERC subfolders and routes matching messages (Appendix L).

All five critical static checks and the critical LLM rubric pass. Empty batch `failed` lists are successful results, not API errors. The trace uses returned folder IDs in later calls.

Q.2 Incomplete Trace: OB-H307 (inbox-cleanup, D3)

The query requests ordered automated/high-importance/unread triage while excluding Sent Items.

The final answer is *Let me continue getting all inbox messages:.* The judgment records incomplete

execution and a missing final count report. The critical LLM failure prevents passing despite reaching the score threshold.

The trace also exposes environment and grading limits: message collection reads remain frozen, folder counts are not a live mutation view, and the static checks do not establish complete ordered triage. Some folder lookups do consult recorded creations. The recorded result supports an incomplete-workflow example.

R Per-Difficulty Breakdown (All Models)

Table 38 extends Table 19 with all eight configurations and additional metrics.

Interpreting assigned difficulty. D1 labels do not guarantee tool availability or easy grading. Some D1 tasks request board operations that the adapter cannot expose. The observed D1/D2/D3 differences therefore cannot be explained solely by model knowledge of rare endpoints.

S Scenarios Failed by the Four-Configuration Cohort

117 scenarios (56.8% of 206) are passed by none of the named Sonnet/Opus/GPT-5.4-R/o3 cohort. Table 39 compares their declared metadata with the full suite.

This cohort includes tasks with unavailable operations, persona mismatches, and grading limitations, as well as potentially difficult model behaviors. Its composition cannot be interpreted as a pure ranking of intrinsic reasoning difficulty.

T Iteration Distribution per Model

Table 40 shows the complete iteration count distribution and summary statistics for each configuration.

Interpretation. Iteration counts include tool-calling steps and terminal answer steps when recorded. A one-iteration trajectory need not be a refusal, and a high count need not be an error-retry loop. Four exception records contain calls despite a zero stored iteration count. Their reconstructed counts are lower bounds.

U Reproducibility Details

Recorded deployments and request settings. Table 16 gives the internal endpoint IDs, run IDs, and timestamps preserved in the eight saved

Level	<i>n</i>	Son.	Opus	5.4-R	5.5-R	5.4	5.2	o3	GPT-5
D1	17	11.8	11.8	5.9	11.8	11.8	5.9	11.8	11.8
D2	121	42.1	38.8	37.2	33.1	31.4	30.6	30.6	28.1
D3	68	23.5	17.6	17.6	22.1	19.1	13.2	10.3	10.3

Table 38: Pass rates (percent) for all eight configurations by assigned difficulty. These groups differ in tasks and tool availability. Differences do not isolate difficulty as a causal factor.

Attribute	Failed by four (117)	All (206)
Mean assigned difficulty	2.27	2.25
Mean declared domains	1.73	1.59
Mean static assertions	1.57	1.25
Mean LLM assertions	1.01	1.02
D3 fraction	40.2%	33.0%
Calendar domain	45.3%	34.5%
Boards domain	11.1%	9.2%
Two or more domains	46.2%	40.8%

Table 39: Scenario metadata for tasks failed by Sonnet, Opus, GPT-5.4-R, and o3. Attributes overlap. This selected four-configuration cohort is not the top four by pass rate.

Model	μ	Med	σ	Max	1	≥ 10	≥ 15
Sonnet 4.5	3.66	3	2.67	15	7	9	6
Opus 4.1	5.52	4	3.91	15	4	33	15
GPT-5.4-R	4.58	3	3.19	15	11	14	7
GPT-5.5-R	4.63	4	2.60	13	6	10	0
GPT-5.4	3.09	2	1.88	12	13	4	0
GPT-5.2	2.33	2	1.77	12	72	4	0
o3	7.31	5	5.10	15	25	66	44
GPT-5	2.35	2	1.52	11	71	1	0

Table 40: Observed iteration distribution. σ is the population standard deviation. Counts in the last three columns use all 206 scenarios. Missing stored counts are replaced by observed lower bounds from call indices.

baseline files. These service routing identifiers do not establish corresponding public model snapshots. All eight agent configurations and the judge use hosted deployments through the same Substrate bridge, including the Anthropic-labeled configurations. The checked-in agent and judge requests omit temperature and specify `max_completion_tokens=4096` and 2048, respectively. Agent and judge bridge subprocess limits are 180 and 120 seconds per request, and the underlying HTTP timeout is 120 seconds. One saved o3-labeled trajectory records 243.575 seconds of agent latency, confirming that the agent limit is not a scenario-wide deadline. Recorded latency excludes subsequent grading calls.

Runtime and cost provenance. The saved artifacts do not establish exact historical workstation or runtime versions, effective hosted endpoint defaults, complete-suite wall-clock duration, or billed

token usage. We therefore make no fixed hardware, runtime-version, or cost claim.

Evaluation Variance. The analysis includes one recorded run per configuration. Repeated-run variance and expert-annotation agreement are not available.

Artifact provenance. Files include model IDs, run IDs, timestamps, final answers, tool traces, and grades. They omit complete source-commit and tool-schema manifests, and tool responses may be truncated. Offline checks reproduce component scores, combined scores, and critical-check pass flags for all 1,648 saved results, but these omissions still limit exact reconstruction of the evaluated environment.

Software Dependencies. The local benchmark uses Node.js, TypeScript, and the repository’s npm dependencies. Model and judge execution additionally requires Python with `requests`, Azure identity and Key Vault libraries, authorized credentials, and access to the configured Substrate service. Recomputing saved-result tables and figures is offline and does not call a model or judge.